

GAMING USING DIFFERENT HAND GESTURES

*Major project report submitted
in partial fulfillment of the requirement for award of the degree of*

**Bachelor of Technology
in
Computer Science & Engineering**

By

T. SWATHI PRATHYUSHA (19UECS0983) (VTU13701)

*Under the guidance of
Mrs. S. Prema, M.E.,
ASSISTANT PROFESSOR*



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF
SCIENCE & TECHNOLOGY**

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

**Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA**

April, 2023

GAMING USING DIFFERENT HAND GESTURES

*Major project report submitted
in partial fulfillment of the requirement for award of the degree of*

**Bachelor of Technology
in
Computer Science & Engineering**

By

T. SWATHI PRATHYUSHA (19UECS0983) (VTU13701)

*Under the guidance of
Mrs. S. Prema, M.E.,
ASSISTANT PROFESSOR*



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF
SCIENCE & TECHNOLOGY**

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

**Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA**

April, 2023

CERTIFICATE

It is certified that the work contained in the project report titled “GAMING USING DIFFERENT HAND GESTURES” by “T. SWATHI PRATHYUSHA (19UECS0983)” has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor

Mrs. S. Prema

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science & Technology

April, 2023

Signature of Head of the Department

Dr. M. S. Murali Dhar

Associate Professor & Head

Computer Science & Engineering

School Of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science & Technology

April, 2023

Signature of the Dean

Dr. V. Srinivasa Rao

Professor & Dean

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science & Technology

April, 2023

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

T. SWATHI PRATHYUSHA

Date: / /

APPROVAL SHEET

This project report entitled “GAMING USING DIFFERENT HAND GESTURES” by T. SWATHI PRATHYUSHA (19UECS0983) is approved for the degree of B.Tech in Computer Science & Engineering.

Examiners

Supervisor

Mrs. S. Prema, M.E.,

Date: / /

Place: Avadi, Chennai.

ACKNOWLEDGEMENT

I express my deepest gratitude to our respected **Founder Chancellor and President Col. Prof. Dr. R. RANGARAJAN B.E. (EEE), B.E. (MECH), M.S (AUTO),D.Sc., Foundress President Dr. R. SAGUNTHALA RANGARAJAN M.B.B.S.** Chairperson Managing Trustee and Vice President.

I am very much grateful to our beloved **Vice Chancellor Prof. S. SALIVAHANAN**, for providing me with an environment to complete my project successfully.

I record indebtedness to our **Professor & Dean, Department of Computer Science & Engineering, School of Computing, Dr. V. SRINIVASA RAO, M.Tech., Ph.D.**, for immense care and encouragement towards me throughout the course of this project.

I am thankful to our **Head, Department of Computer Science & Engineering, Dr.M.S. MURALI DHAR, M.E., Ph.D.**, for providing immense support in all my endeavors.

I also take this opportunity to express a deep sense of gratitude to my Internal Supervisor **Mrs. S. PREMA, M.E.**, for her cordial support, valuable information and guidance, she helped me in completing this project through various stages.

A special thanks to our **Project Coordinators Mr. V. ASHOK KUMAR, M.Tech., Ms. C. SHYAMALA KUMARI, M.E.**, for their valuable guidance and support throughout the course of the project.

I thank our department faculty, supporting staff and friends for their help and guidance to complete this project.

T. SWATHI PRATHYUSHA (19UECS0983)

ABSTRACT

Hand gesture recognition system that can identify various hand gestures, including pointing, waving, thumbs up/down, fist, and open hand in real-time using a camera input. The system should be resilient to variations in lighting, camera orientation, and background. It should also be able to recognize and track multiple hands in the same frame, providing real-time output for controlling devices or interacting with computer interfaces. The way people engage with the system may be constrained by traditional interface methods like the keyboard, mouse, touchscreen, etc. To communicate with any of these systems, physical contact is necessary. The same functionality can be interpreted by gestures without requiring physical contact with the interfaced devices. The system's primary objective is to show users how to use hand gestures to control a computer game. The secondary objective of this system is to create a system that allows a player to play a game without a physical controller. The system seeks to create a gesture recognition application. The camera or webcam connected to the system can be used to recognize the human hand gesture. Based on the analysis made by the system on recognizing the human Hand Gestures, the operations on the Game will be performed with the game's default gaming controls. The gestures are to be done on the palms of the hands. The system will include two modules: recognition of gestures and analysis of gestures. The gestures recognition module can be used to recognize the gestures. At last, the analysis module will analyze the human hand gesture and perform game controls based on the calculated analysis. People with disabilities can also use these hand signals to communicate with the systems.

Keywords: Hand gestures; Physical controller; Gesture recognition; Gaming controls; Analysis of gestures.

LIST OF FIGURES

4.1	Architecture Diagram of Hand Gesture System	11
4.2	Data Flow Diagram of Hand Gesture System	12
4.3	Use Case Diagram of Hand Gesture System	13
4.4	Class Diagram of Hand Gesture System	14
4.5	Sequence Diagram of Hand Gesture System	15
4.6	Collaboration Diagram of Hand Gesture System	16
4.7	Activity diagram of Hand Gesture System	17
4.8	21 3D landmarks of Hand	20
4.9	List of Libraries	22
4.10	Initializing Holistic Model	22
5.1	Input Image for Hand Gesture System	24
5.2	Output Image for Hand Gesture System	25
5.3	Unit Testing Result of Hand Gesture System	26
5.4	Integration Testing Result of Hand Gesture System	29
5.5	System Testing Result of Hand Gesture System	31
6.1	Comparison Table for Existing and Proposed System	32
6.2	Output when Gas Button is operated	36
6.3	Output when Brake Button is operated	37
8.1	Internship Offer Letter	40
8.2	Commencement Form	41
8.3	Internship Completion Certificate	42
9.1	Plagiarism Report	43
10.1	Poster Presentation of Hand Gesture System	50

LIST OF ACRONYMS AND ABBREVIATIONS

ANN	Artificial Neural Networks
BGR	Blue, Green, Red
CC	Computer Control
DFD	Data Flow Diagram
HCI	Human Computer Interaction
NUI	Natural User Interface
RGB	Red, Green and Blue
YOLO	You Only Look Once

TABLE OF CONTENTS

	Page.No
ABSTRACT	v
LIST OF FIGURES	vi
LIST OF TABLES	vii
LIST OF ACRONYMS AND ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Project Domain	2
1.4 Scope of the Project	2
2 LITERATURE REVIEW	3
3 PROJECT DESCRIPTION	6
3.1 Existing System	6
3.2 Proposed System	7
3.3 Feasibility Study	8
3.3.1 Economic Feasibility	8
3.3.2 Technical Feasibility	9
3.3.3 Social Feasibility	9
3.4 System Specification	9
3.4.1 Hardware Specification	9
3.4.2 Software Specification	10
3.4.3 Standards and Policies	10
4 METHODOLOGY	11
4.1 General Architecture	11
4.2 Design Phase	12
4.2.1 Data Flow Diagram	12

4.2.2	Use Case Diagram	13
4.2.3	Class Diagram	14
4.2.4	Sequence Diagram	15
4.2.5	Collaboration Diagram	16
4.2.6	Activity Diagram	17
4.3	Algorithm & Pseudo Code	18
4.3.1	Algorithm	18
4.3.2	Pseudo Code	18
4.4	Module Description	19
4.4.1	Pose Estimation	19
4.4.2	Preprocessing	20
4.4.3	Feature Extraction	21
4.4.4	Segmentation	21
4.5	Steps to execute/run/implement the project	22
4.5.1	Import Libraries	22
4.5.2	Initializing Model	22
4.5.3	Implementation	23
5	IMPLEMENTATION AND TESTING	24
5.1	Input and Output	24
5.1.1	Input Design of Hand Gesture System	24
5.1.2	Output Design of Hand Gesture System	25
5.2	Testing	25
5.3	Types of Testing	25
5.3.1	Unit Testing - Pose Estimation	25
5.3.2	Integration Testing - Preprocessing	27
5.3.3	System Testing - Segmentation	30
6	RESULTS AND DISCUSSIONS	32
6.1	Efficiency of the Proposed System	32
6.2	Comparison of Existing and Proposed System	32
6.3	Sample Code	34
7	CONCLUSION AND FUTURE ENHANCEMENTS	38
7.1	Conclusion	38
7.2	Future Enhancements	38

8	INDUSTRY DETAILS	39
8.1	Industry name	39
8.1.1	Duration of Internship (From Date - To Date)	39
8.1.2	Duration of Internship in months	39
8.1.3	Industry Address	39
8.2	Internship offer letter	40
8.3	Project Commencement Form	41
8.4	Internship Completion Certificate	42
9	PLAGIARISM REPORT	43
10	SOURCE CODE & POSTER PRESENTATION	44
10.1	Source Code	44
10.2	Poster Presentation	50
	References	51

Chapter 1

INTRODUCTION

1.1 Introduction

Over the past ten years, computer technology has advanced significantly and has become an essential element of daily life. The keyboard is the most important computer accessory for Human Computer Interaction (HCI). The keyboard is inappropriate. The most natural and intuitive technique for HCI, that is a viable replacement for the computer keyboard is with the use of hand gestures. Gaming with the Computer accessories like keyboard, mouse, joystick, etc. has always been a repetitive thing for gamers and when for long hours of usage, it becomes a boring task. This ultimately affects in the game company's loss as gamers get fed up of all this. In order to harness the full potential of a webcam, can be used for vision-based CC, which would effectively eliminate the need for a computer keyboard. Significant improvements in HCI gaming technology have been made during the past few decades. Motion controllers are the way of the future of gaming, and they have significantly increased the sales of video games, such as the Nintendo Wii, which sold more than 50 million consoles in its first year of availability. A Natural User Interface (NUI) is provided by HCI employing hand gestures, which is particularly intuitive and efficient for one-on-one computer contact. Many studies have been done on new tools and methods for hand motions to control the cursor. The use of hand gesture recognition in sign language recognition and HCI raises the importance of the technology even further.

1.2 Aim of the Project

The project is mainly aimed at investigating and developing a Computer Control (CC) system using hand gestures. The main aim of the project is to create a interactive platform for using laptop or PC which can be controlled using hand gestures. This project is to make the gaming platform more user interactive and interesting.

1.3 Project Domain

HUMAN-COMPUTER INTERACTION

The study of human-computer interaction (HCI) focuses on the user interfaces that people (users) employ to interact with computers. Human-computer interaction (HCI) researchers examine how people use computers and create solutions that enable people to use them in novel ways. A Human-computer Interface (HCI) is a device that makes it possible for people and computers to communicate. Research in the field of human-computer interaction crosses several academic fields, including computer science, psychology, design, media studies, and behavioural sciences. In their 1983 book *The Psychology of Human-Computer Interaction*, Stuart K. Card, Allen Newell, and Thomas P. Moran popularised the phrase. In 1975, Carlisle made use of it for the first time.

The phrase is meant to express that, unlike other technologies with narrowly defined purposes, computers have a wide range of applications that frequently entail an ongoing conversation between the user and the computer. The idea of dialogue compares human-computer interaction to human-to-human interaction, which is a comparison that is essential to the field's theoretical deliberations.

1.4 Scope of the Project

The project deals with the problem of developing a vision based static hand gesture recognition System. The system recognizes different hand gestures performed to replace the left, right, down, up arrows using the various feature extraction methods and Machine learning algorithm for classification of the image captured through the webcam and OpenCV is used to capture the images from a live streaming video.

Chapter 2

LITERATURE REVIEW

[1] K. Akula Gokul, et al, [2022] proposed that project aims to create a unique application that connects to the game. Input and interaction devices are an important part of every game for human interaction but in recent times it has been developed with various technologies to control computer-based systems. Interaction between humans comes from different modes like gestures, speech, text etc. All gesture interaction is best for interacting with games. Furthermore, the approach of stochastic gradient descent hand gesture recognition system can efficiently track both static and dynamic hand gestures.

[2] Tanay Thakor, et al, [2021] proposed that the system aims at developing an application, for gesture recognition. The camera or webcam connected to the system can be used to recognize the human hand gesture. Based on the analysis made by the application done on recognizing the human Hand Gestures, the operations on the Game will be performed with the game's default gaming controls. The application contains a set of instruction to recognize the human hand gestures. The gestures are to be done on the palms of the hands. A completely robust hand gesture recognition system is still under heavy research and development. This system work can serve as small step towards an extensible foundation for future work.

[3] Ahmed S. Khalaf, et al, [2020] proposed a simple system of hand-gesture-based game to evaluate performance, cognitive demand, comfort, and player experience of

using these gesture devices. System found that participants preferred and performed much better using Leap Motion and Kinect compared to using RealSense. Leap Motion also outperformed or was equivalent to Kinect. These findings were supported by players' accounts of their experiences using these gesture devices. Based on these findings, It is discussed how such devices can be used by game designers and provide them with a set of design cautions that provide insights into the design of gesture-based games.

[4]Diego Perez-Liebana, et al, [2019] proposed that everybody deserve to live independently, especially those who disabled, with the last decades, technology gives attention to disabled to make them control their life as much as possible. In this work, an assistive system for blind is suggested, to let him know what is around him, by using YOLO for detecting objects within images and video streams quickly based on deep neural networks to make accurate detection, and OpenCV under Python using Raspberry Pi3. The obtained results indicated the success of the proposed model in giving blind users the capability to move around in unfamiliar indoor-outdoor environments, through a user-friendly device by person and object identification model.

[5] Nagashree, et al, [2018] proposed object detection is a well-known computer technology connected with computer vision and image processing that focuses on detecting objects or its instances of a certain class (such as humans, flowers, animals) in digital images and videos. There are various applications of object detection that have been well researched including face detection, character recognition, and vehicle calculator. Object detection can be used for various purposes including retrieval and surveillance. In this study, various basic concepts used in object detection while making use of the OpenCV library of python 2.7.

[6]D. Perez, et al., [2015] proposed an object detection system that finds objects of the real-world present either in a digital image or a video, where the object can belong to any class of objects namely humans, cars, etc. In order to detect an object in an image or a video the system needs to have a few components in order to complete the task of detecting an object, they are a model database, a feature detector, a hypothesis and a hypothesis verifier. This paper presents a review of the various techniques that are used to detect an object, localize an object, categorize an object, extract features, appearance information, and many more, in images and videos.

[7]M. Swiechowski et al, [2015] presented object detection and tracking as one of the critical areas of research due to routine change in motion of object and variation in scene size, occlusions, appearance variations, and ego-motion and illumination changes. Specifically, feature selection plays a vital role in object tracking. It is related to many real time applications like vehicle perception, video surveillance and so on. In order to overcome the issue of detection, tracking related to object movement and appearance. Most of the algorithm focuses on the tracking algorithm to smoothen the video sequence. On the other hand, few methods use the prior available information about object shape, color, texture and so on. Tracking algorithm which combines the above stated parameters of objects is discussed and analyzed in this research.

Chapter 3

PROJECT DESCRIPTION

3.1 Existing System

The existing system for real-time applications such as digital games, the use of input devices has become a strong barrier to communicate with the computer. So, this new hand-gestures recognition is introduced where humans are integrated with computer through hand gestures. The hand gestures that form the main input are pre-processed, backgrounds are removed from the images, noise filtered using a K means algorithm to find the exact position of the fingers. Refined finger positioning used to control digital games. The system translates the detected gesture into actions such as playing games. For testing purpose, applied on Rock, Paper and Scissors, Hill Climb Racing, Off-Road Super Racing games. The games are controlled using static and dynamic gestures. Dynamic gesture is used to change controls in games. The results show that human computer interaction can be achieved even with minimal hardware requirements.

Disadvantages of Existing System

- This System has the main disadvantage as it can not detect the hand when it is in continuous motion.
- After the gesture is showed to the webcam, it still takes along time to detect the gesture and perform the action.

- The setup OpenCV for our IDE (Python 3x Idle) is very difficult and should have minimum knowledge about OpenCV.
- Another main disadvantage is to remember the hand gestures recognized as the replacement of keys.

3.2 Proposed System

The proposed system deals with the problem of developing a vision based static hand gesture recognition algorithm to recognize the following four static hand gestures:left, right, up, down arrows. These gestures are chosen because they are commonly used to communicate and can thus be used in various applications, such as, a virtual Keyboard for a given application. The proposed system consists mainly of three phases: the first phase (i.e., pre-processing), the next phase (i.e., feature extraction) and the final phase (i.e., classification).

The first phase includes hand segmentation that aims to isolate hand gesture from the background and removing the noises using special filters. This phase includes also edge detection to find the final shape of the hand. The next phase, which constitutes the main part of this research, is devoted to the feature extraction problem where two feature extraction methods, namely, hand contour and complex moments are employed. These two extraction methods were applied in this study because they used different approaches to extract the features, namely, a boundary-based for hand contour and region-based for complex moments.

The feature extraction algorithms deal with problems associated with hand gesture recognition such as scaling, translation and rotation. As a classification method, ANN has been widely employed especially for real-world applications because of its ability to work in parallel.

Advantages of proposed System

- This System improved the performance by 22% compared to existing time.
- The duration of hand gesture recognition is reduced to the 3 seconds and it is fast and accurate.
- People who are physically handicapped (without fingers) can also play the game with the help images of the hand gestures.
- Paralyzed or hospitalized people can also play the games for entertainment purpose.
- Children feel more interested in playing games with the help of their and gestures instead of using keyboard or mouse.

3.3 Feasibility Study

3.3.1 Economic Feasibility

The Economic Feasibility involves Developing a hand gesture recognition system for gaming requires a significant investment in terms of software development, hardware, and testing. The cost of development should be weighed against the potential revenue generated by the game. The market size for gesture-based games needs to be evaluated to determine the potential demand for the game.

The target audience and their willingness to pay for such games should also be taken into consideration. The competition in the gaming industry is intense, and there are many games available on various platforms. The game should have a unique selling point and should be able to differentiate itself from other games available on the market.

3.3.2 Technical Feasibility

The technology required for hand gesture recognition, such as cameras or sensors, must be available and reliable. The algorithms used for recognizing hand gestures must be accurate and able to handle various lighting conditions and hand positions. The system must be able to recognize a wide range of hand gestures, depending on the application. The system should be compatible with the device on which the game will be played, such as a PC, gaming console, or mobile device. The software used for hand gesture recognition should be able to accurately recognize different hand gestures and translate them into game commands.

3.3.3 Social Feasibility

The hand gesture recognition system should be accessible to all players, including those with disabilities or limitations that affect their ability to use traditional game controllers. The hand gesture recognition system should be designed to be inclusive of all genders, races, and cultures. The system should not rely on gestures that are specific to one group or culture, and the game should not perpetuate harmful stereotypes. The hand gesture recognition system should protect the privacy and security of the players. The system should not collect or share personal data without the player's consent.

3.4 System Specification

3.4.1 Hardware Specification

- Device Name:- HP
- Processor:-Intel Core i7
- Installed RAM:-16.00GB

- System type:-64-bit operating system,x64-based processor

3.4.2 Software Specification

- OpenCV Version- 4.7.0
- Mediapipe 0.9.1.0 version
- Visual code Version 1.76
- Python version 3.7-3.11 (Except 3.10)

3.4.3 Standards and Policies

OpenCV:OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications. **Standard Used: Version 4.7.0**

MEDIAPIPE: Media Pipe is a Framework for building machine learning pipelines for processing time- series data like video, audio, etc. This cross-platform Framework works in Desktop/Server, Android, iOS, and embedded devices like Raspberry Pi and Jetson Nano. **Standard Used: Version C++11**

PYTHON: Python technology is quite popular among programmers, It was created by Guido van Rossum and released in 1991. version of Python for Media Pipe is To make Media pipe work with TensorFlow, please set Python 3.7 as the default Python version.

VISUAL CODE: Visual Studio empowers you to complete the entire development cycle in one place. For example, you can edit, debug, test, version control, and deploy to the cloud. With the diversity of features and languages in Visual Studio, you can grow from writing your first piece of code to developing in multiple project types. **Standard Used: Version 1.7**

Chapter 4

METHODOLOGY

4.1 General Architecture

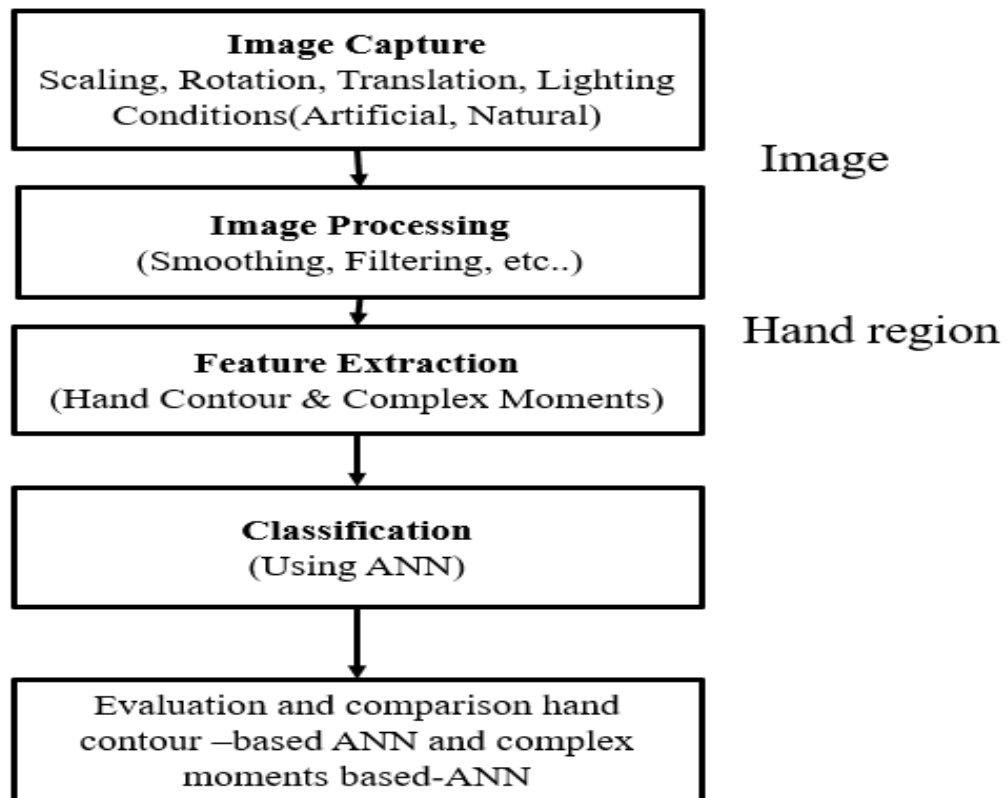


Figure 4.1: Architecture Diagram of Hand Gesture System

In Figure 4.1, Architecture Diagram of Hand Gesture System tells about the steps involved in the hand gesture recognition system. The first step is to capture the image form the live streaming image. The next step is Image processing where the image is smoothed and filtering and noise are removed. Feature extraction is the next step where complex hand moments and the features of image are identified. Following step is Classification where the number of fingers are identified and classified. Final

step is the evaluation of hand contour based ANN and complex moments based ANN and finally result is displayed.

4.2 Design Phase

4.2.1 Data Flow Diagram

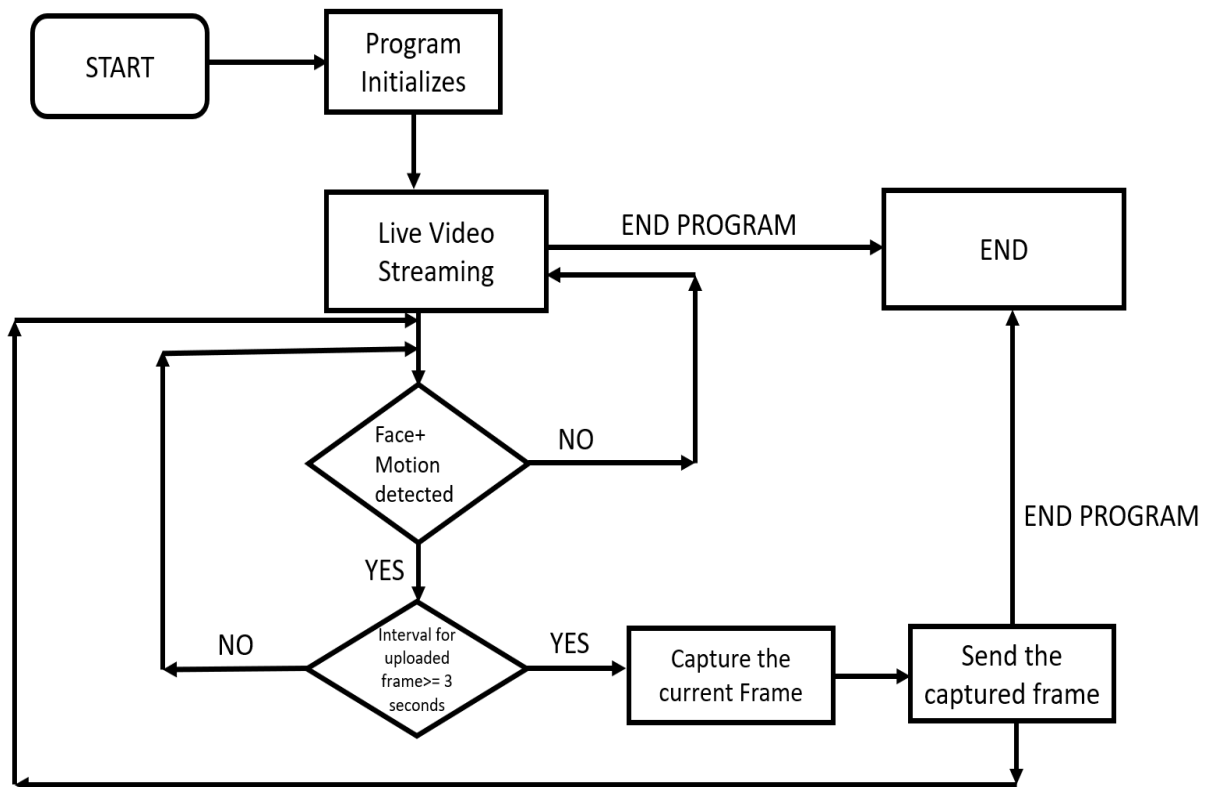


Figure 4.2: Data Flow Diagram of Hand Gesture System

In Figure 4.2, Data Flow Diagram of Hand Gesture System maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination. After the code is run then the program initializing is done and frames are detected if any hand gesture is shown more than three seconds, it is considered as input and the output action is performed to display the output.

4.2.2 Use Case Diagram

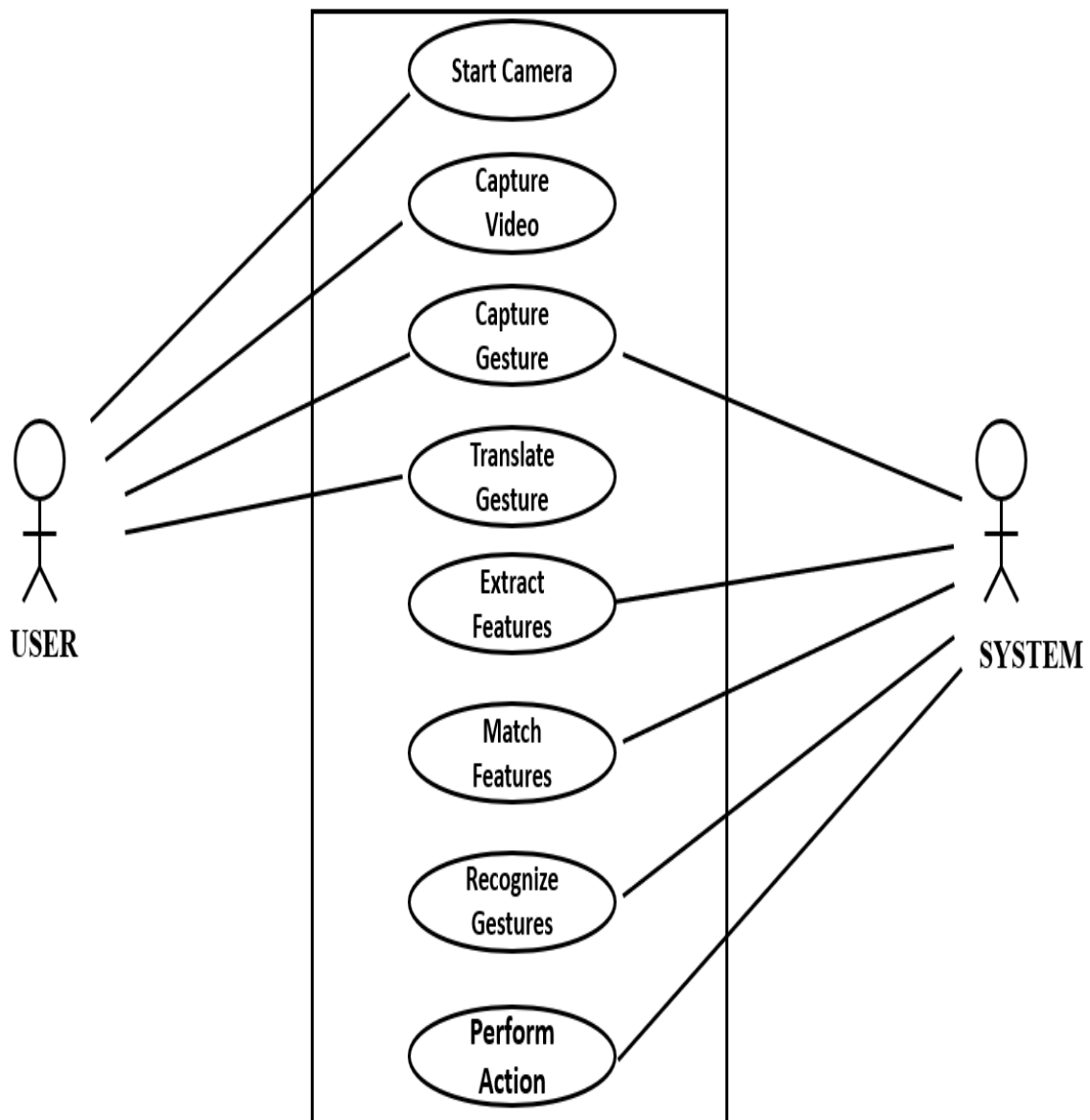


Figure 4.3: Use Case Diagram of Hand Gesture System

In Figure 4.3, Use case Diagram of Hand Gesture System identify the interactions between the system and its actors. The above diagram shows the actions to be performed by the user of this system like starting the camera, capturing video, capturing the gestures, translating the gesture and system actions are capturing the gesture, extract the features, match the features, recognize gestures and perform the actions.

4.2.3 Class Diagram

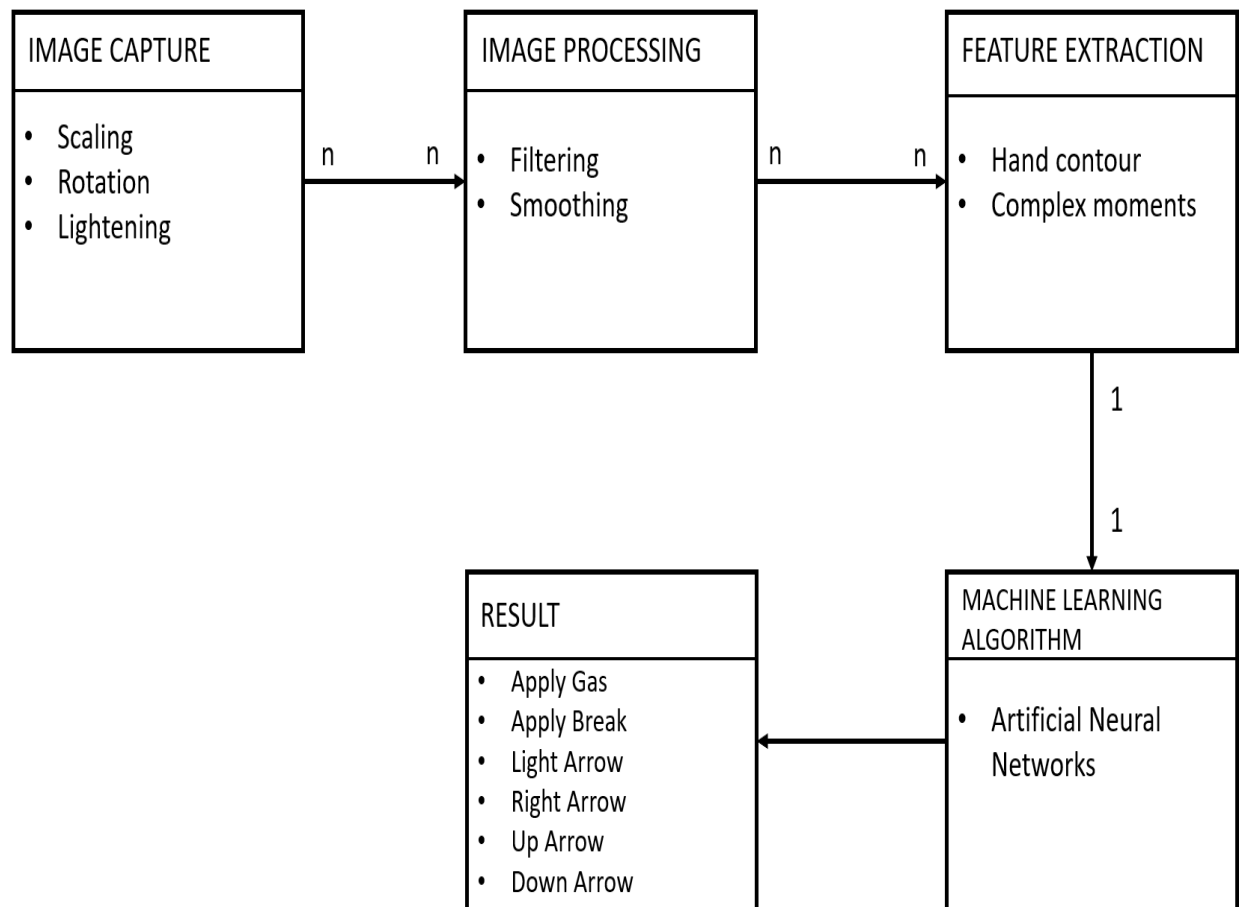


Figure 4.4: Class Diagram of Hand Gesture System

In Figure 4.4, A Class Diagram of Hand Gesture System is a diagram used in designing and modeling software to describe classes and their relationships. The main Classes in this diagram are image capture, image processing, feature extraction, machine learning algorithms for classification, result. The captured image will undergo the scaling, translation and image will be processed using filtering and smoothing. feature extraction methods are used and the machine learning algorithms used are the artificial neural networks and the result is shown with the gas, brake, up, left, right, down arrows.

4.2.4 Sequence Diagram

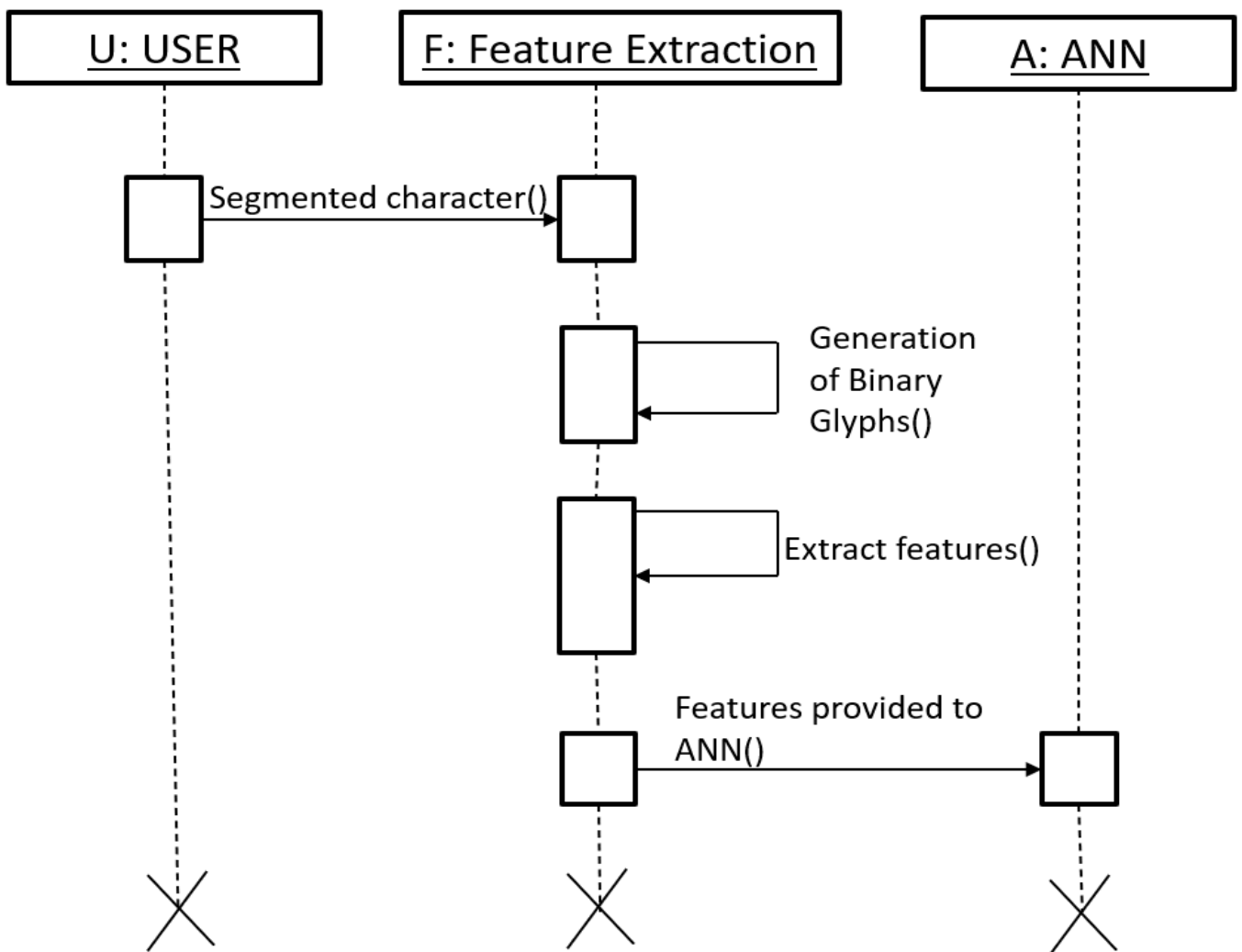


Figure 4.5: Sequence Diagram of Hand Gesture System

In Figure 4.5, A Sequence Diagram of Hand Gesture System shows the sequence of messages passed between objects. Sequence diagrams can also show the control structures between objects. The communication done between the user and feature extraction is segmented character is passed in between and the feature extraction communications are the generation of the glyph, Extract the features and the last is features extracted are provided to the ANN from the feature extraction.

4.2.5 Collaboration Diagram

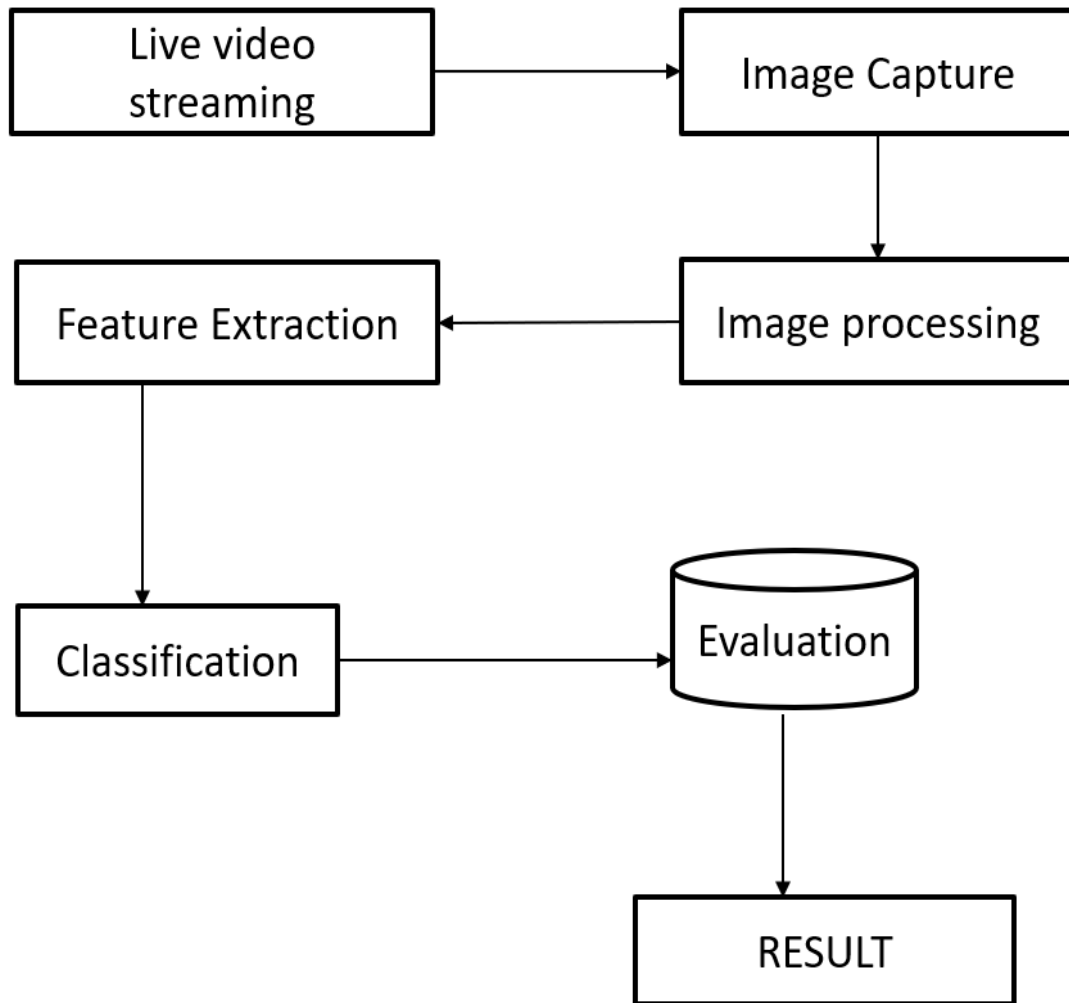


Figure 4.6: Collaboration Diagram of Hand Gesture System

In Figure 4.6, Collaboration diagram of Hand Gesture System refers proposed system consists mainly of three phases. The The first phase includes hand segmentation that aims to isolate hand gesture from the background and removing the noises using special filters. The feature extraction algorithms deal with problems associated with hand gesture recognition such as scaling, translation and rotation. In the classification phase where neural networks are used to recognize the gesture image based on its extracted feature, analyze some problems related to the recognition and convergence of the neural network algorithm.

4.2.6 Activity Diagram

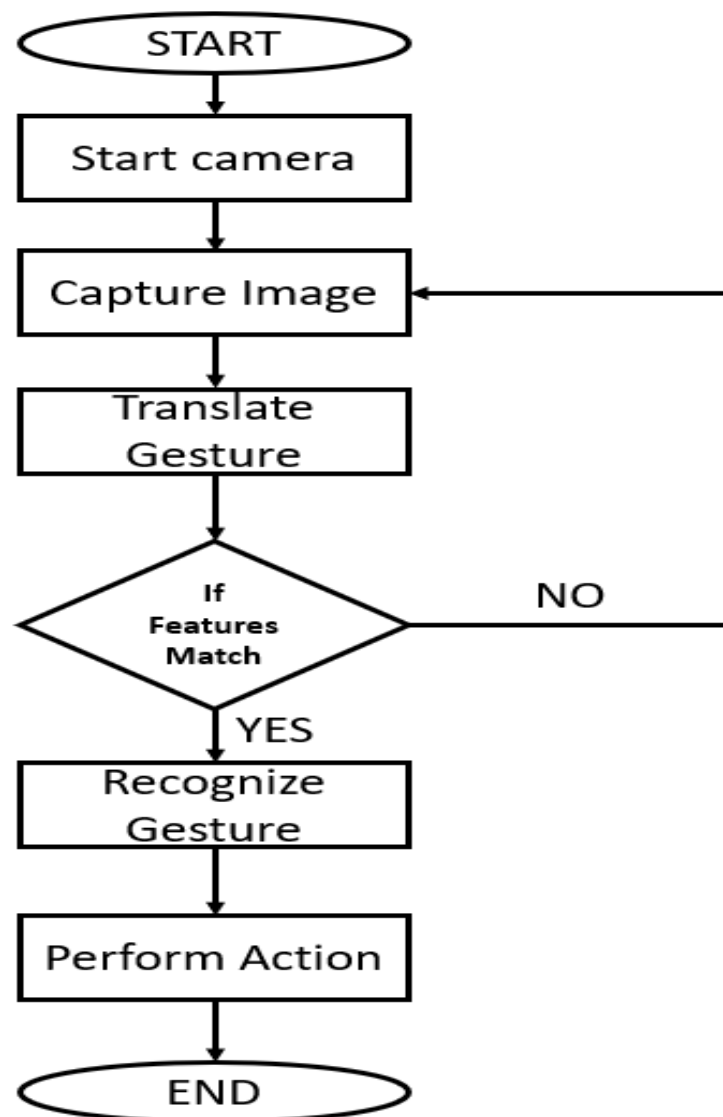


Figure 4.7: Activity diagram of Hand Gesture System

In figure 4.7, The Activity diagram of Hand Gesture System shows the flow of activities and processes in the hand gesture recognition system. The camera starts running and image is captured. The captured image if it matches the features of pre-defined gestures the action is performed and if not then the process will go back to again capturing the image. If the image is captured then matching process starts and if not the process repeats continuously.

4.3 Algorithm & Pseudo Code

4.3.1 Algorithm

Step 1: Open Visual Studio Code.

Step 2: Install the necessary libraries: OpenCV, Mediapipe, and Python.

Step 3: Start writing the Code in the Python file.

Step 4: Import the required libraries.

Step 5: Create a video capture object.

Step 6: Initialize the hand tracking module from Mediapipe.

Step 7: Create a drawing utility for the landmarks detected by Mediapipe.

Step 8: Start the main loop to read frames from the video capture object.

Step 9: Perform hand gesture recognition using the landmarks obtained from step 8:

Step 10: You can use a machine learning algorithm (Artificial Neural Networks) for this task.

Step 11: Display the output on the screen.

Step 12: Release the video capture object and destroy any open windows.

4.3.2 Pseudo Code

Step 1: Import the necessary libraries.

Step 2: Create a video capture object.

Step 3: Initialize the hand tracking module from Mediapipe.

Step 4: Create a drawing utility for the landmarks detected by Mediapipe.

Step 5: Start the main loop to read frames from the video capture object.

Step 6: Read a frame from the video capture object.

While true:

 Step 6.1: Convert the frame to RGB format for processing by Mediapipe.

Step 6.2: Process the frame using Mediapipe.

Step 6.3: Check if any hand is detected in the frame and get the landmarks.

Step 7: Perform hand gesture recognition using the landmarks obtained from above.

Step 8: You can use a machine learning algorithm or a rule-based system for this task.

Step 9: Display the output on the screen.

Step 10: Release the video capture object and destroy any open windows.

4.4 Module Description

4.4.1 Pose Estimation

Posture estimation is a crucial component of hand gesture recognition since it aids in identifying and following the major points or landmarks of the hand. For recognising hand gestures, Media-pipe offers a posture estimation package for Python. Human pose estimation from video or a real-time feed plays a crucial role in various fields such as full-body gesture control, quantifying physical exercise, and sign language recognition.

Media Pipe Pose is a framework for high- fidelity body pose tracking, which takes input from RGB video frames and infers 21 3D landmarks on the whole human hand. Current state-of-the-art approach methods rely primarily on powerful desktop environments for inferencing, whereas this method outperforms other methods and achieves very good results in real-time. You can use machine learning algorithms to recognize hand gestures using the landmarks obtained from Mediapipe's pose estimation module.

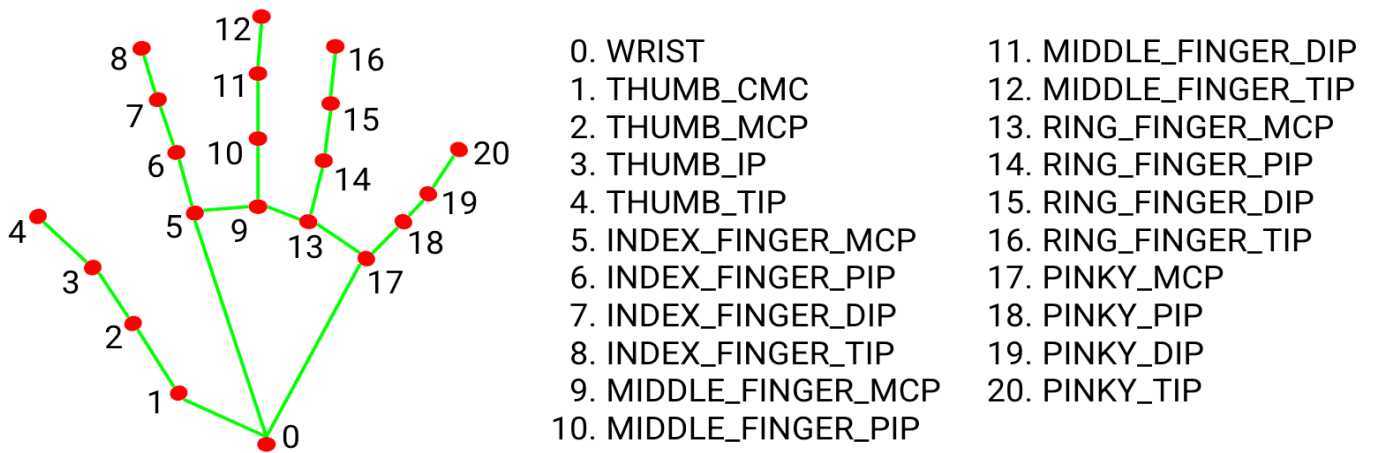


Figure 4.8: **21 3D landmarks of Hand**

In Figure 4.8, 21 3D landmarks of Hand is the figure that shows the landmarks of the hand marked on the entire hand. The numbering of the landmark starts from 0 and ends with 20 and hence the complete human hand has 21 3D landmarks on the hand to play game using hand gestures. According to the landmarks identified and then the shape of the hand shown to the camera is finalized and edges of the hand are recognized and action is performed.

4.4.2 Preprocessing

Preprocessing is a crucial module in the recognition of hand gestures since it helps to improve the quality of the input data, reduce noise, and increase the precision of the recognition algorithm. For the purpose of recognising hand gestures, Python's OpenCV library offers a variety of image preprocessing tools. In this Phase the BGR image is converted to RGB image. The system's computational complexity can be decreased by converting the image, which can also increase recognition precision. In this Preprocessing to improve the process performance, prepared a model or image as not writable. As completed or trained the model then made a model or image as writable and covert back the RGB image to BGR image.

4.4.3 Feature Extraction

In order to extract relevant data from the input image that can be used for classification, feature extraction is a crucial module in hand gesture recognition. For feature extraction in Python, OpenCV offers a number of functions that can be utilised for hand gesture recognition. In pattern recognition in image processing, Feature extraction is a special form of dimensionality reduction.

When the input data to an algorithm is too large to be processed and it is suspected to be notoriously redundant (much data, but not much information) then the input data will be transformed into a reduced representation set of features (also named feature vector). Transforming the input data into the set of features is called features extraction. In this Module the features(height, width) of the hand shown to camera are analyzed. Some other features can be counting the numbers of fingers and coordinates of the image.

4.4.4 Segmentation

Segmentation is a key module in hand gesture recognition. In order to separate the hand from the background and extract information that can be utilised for classification. OpenCV for Python has a number of segmentation routines that can be used for hand gesture recognition. The segmented hand can be used classification. In this part the characters will be identified as letters and the image will be converted to text. After the image is cleaned up and becomes a binary image which contains only the text, the binary image is then saved, and the memory is cleaned up.

This step is very important to increase the speed of the system. after segmentation classification is carried out. In the classification phase ANN is used for classification where neural networks are used to recognize the gesture image based on its extracted feature, analyze some problems related to the recognition and convergence

of the neural network algorithm. As a classification method, ANN has been widely employed especially for real-world applications because of its ability to work in parallel and online training.

4.5 Steps to execute/run/implement the project

4.5.1 Import Libraries

- Import all the necessary libraries, In our case only three libraries are required.

```
import mediapipe as mp  
import cv2  
import pydirectinput
```

Figure 4.9: **List of Libraries**

4.5.2 Initializing Model

- Initializing Holistic model and Drawing utils for detecting and drawing landmarks on the image.

```
mp_draw=mp.solutions.drawing_utils  
mp_holistic=mp.solutions.holistic  
mp_pose=mp.solutions.pose
```

Figure 4.10: **Initializing Holistic Model**

4.5.3 Implementation

- Detecting Face and Hand landmarks from the image. Holistic model processes the image and produces landmarks for Face, Left Hand, Right Hand and also detects the Pose of the hand move.
- Capture the frames continuously from the camera using OpenCV.
- Will take a live feed from the webcam by changing the first line in the main function to “cap = cv2.VideoCapture(0)”.
- Convert the BGR image to an RGB image and make predictions using an initialized holistic model.
- The predictions made by the holistic model are saved in the results variable from which can be accessed the landmarks using results.facelandmarks, results.righthandlandmarks, results.lefthandlandmarks respectively.
- Draw the detected landmarks on the image using the drawlandmarks function from drawing utils.
- Use the resulting Image as pose movement according to the axis drawing on the image window.

Chapter 5

IMPLEMENTATION AND TESTING

5.1 Input and Output

5.1.1 Input Design of Hand Gesture System



Figure 5.1: Input Image for Hand Gesture System

In figure 5.1, Input Image for Hand Gesture System shows that the button present in hill climbing race (Brake) is operated through the gesture of closed palm. The registered gesture for performing the action of brake button is closed palm. So, When the closed palm is shown to the camera the palm is recognized and the action of brake button is performed. There are different buttons present in the game like brake, gas. Where both the buttons are replaced by the hand gestures.

5.1.2 Output Design of Hand Gesture System



Figure 5.2: Output Image for Hand Gesture System

In figure 5.2, Output Image for Hand Gesture System describes that the hand gesture to operate brake button is captured and in this figure the action is performed through operating the brake button. The registered gesture for performing the action of brake button is closed palm. So, When the closed palm is shown to the camera the palm is recognized and the action of brake button is performed. Hence the game can be played using different hand gestures. The game is played here with the hand gestures.

5.2 Testing

5.3 Types of Testing

5.3.1 Unit Testing - Pose Estimation

Input

```
1 import cv2
2 import mediapipe as mp
3 import time
4 from directkeys import right_pressed, left_pressed, up_pressed, down_pressed
5 from directkeys import PressKey, ReleaseKey
6
7
8 brake_key_pressed=left_pressed
9 accelerate_key_pressed=right_pressed
10 jump_key_pressed=up_pressed
11 slide_key_pressed=down_pressed
```

```

12
13 time.sleep(2.0)                                # suspends execution to no. of sec
14 current_key_pressed = set()                     # build an unordered collection of unique elements assigns to
    a variable
15
16 mp_draw=mp.solutions.drawing_utils             # solution to drawing utils as we are drawing utils
17 mp_hand=mp.solutions.hands                     # solution to hand pose
18
19
20 tipIds=[4,8,12,16,20]

```

Test result

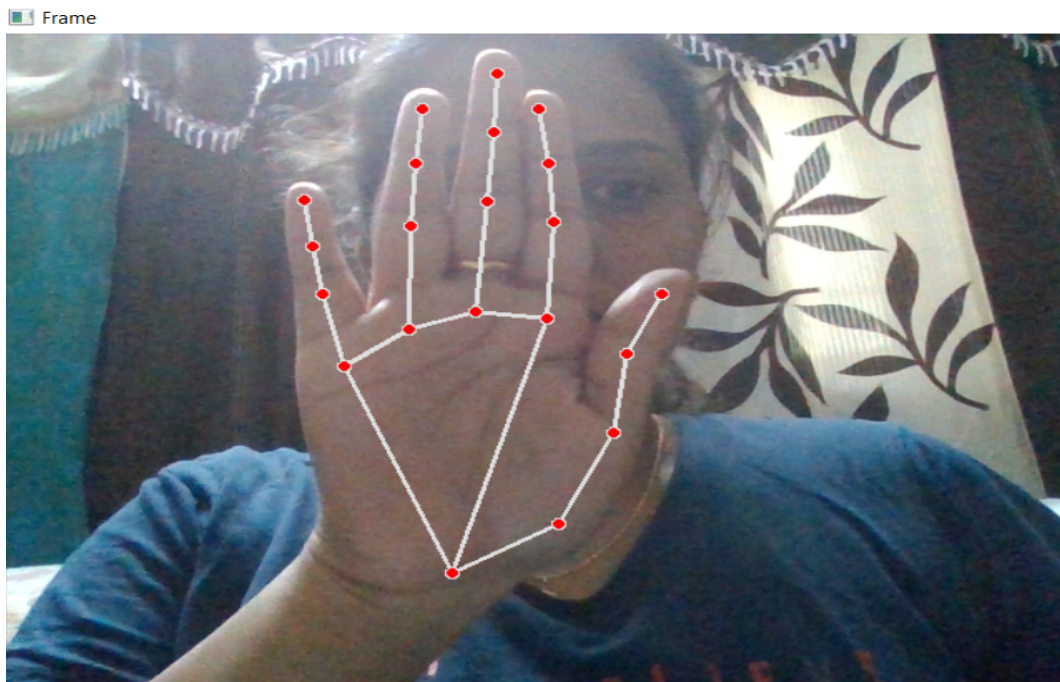


Figure 5.3: Unit Testing Result of Hand Gesture System

In figure 5.3, Unit testing result of Hand Gesture Recognition System describes that Unit testing is done in the above image. It is tested whether all the finger tips of a palm are identified or not. In the figure we are able to identify all Finger tips and according to the landmarks of hand 4, 8, 12, 16, 20 are the tips of the fingers. All the tips are identified in above image. This figure has all the 21 landmarks on the hand which are marked.

5.3.2 Integration Testing - Preprocessing

Input

```
1 mp_draw=mp.solutions.drawing_utils      # solution for drawing utils as we are drawing utils
2 mp_hand=mp.solutions.hands              # solution for hand pose
3
4
5 tipIds=[4,8,12,16,20]                   # List of landmarks of tip of fingers
6
7 video=cv2.VideoCapture(0)               # Start capturing video from webcam, 0 refers to computer or
8     default camera
9 ### Hands fuction of mp processes RBG image amd returns hand landmarks and handedness of detected
10    hand
11 with mp_hand.Hands(max_num_hands=1,min_detection_confidence=0.5,          # max_num_hands = no. of
12    hands to be detected at a time
13    min_tracking_confidence=0.5) as hands:          # min_detection_confidence
14    = min confidence value for detected person-model to be considered as successful.
15    # min_tracking_confidence =
16    while True:
17        min confidence value with which the detection from the landmark-tracking model must be
18        considered as successful
19        keyPressed = False
20        brake_pressed=False
21        accelerator_pressed=False
22        jump_pressed=False
23        slide_pressed=False
24        key_count=0
25        key_pressed=0
26        ret,image=video.read()          # Read video frame by frame & ret checks
27        whether camera is working(1) or not(0)
28        image=cv2.cvtColor(image, cv2.COLOR_BGR2RGB )          # Convert BGR image to RGB image as opencv
29        gives us BGR image
30        #image = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY )
31        image.flags.writeable=False          # To improve the process performance we make
32        model or image as not wrritable
33        results=hands.process(image)          # Process the RGB image only
34        image.flags.writeable=True          # As we completed or trained the model we
35        make model or image as wrritable
36        image=cv2.cvtColor(image, cv2.COLOR_RGB2BGR )          # Convert RGB image to BGR image
37        lmList=[]          # Empty list to store landmarks
38        ### Prediction made by the model is saved in the results variable from which we can access
39        landmarks using results.multi_hand_landmarks
40        if results.multi_hand_landmarks:          # If hands are present in image(frame)
41            for hand_landmark in results.multi_hand_landmarks:          # Iterate all landmarks in
42                variable results
43                myHands=results.multi_hand_landmarks[0]          # provides starting point
44                of hand landmark i.e. 0
45                for id, lm in enumerate(myHands.landmark):          # Enumerate through all ids
46                    and landmarks to provide x,y,z coordinates of each landmark on image
47                    h,w,c=image.shape          # finds height and width of the image and
48                    assigns to h and w
49                    cx,cy= int(lm.x*w), int(lm.y*h)          # finds actual integer coordinates of
50                    landmarks and store them in cx and cy
51                    lmList.append([id,cx,cy])          # Stores each landmark values and its
52                    corresponding coordinates in an empty list one by one
53                    mp_draw.draw_landmarks(image, hand_landmark, mp_hand.HAND_CONNECTIONS) #
54                    draw_landmarks function is used to draw hand landmarks and connections on image
55                    fingers=[]          # list to count no. of fingers visible to
56                    cam at a time
57                    if len(lmList)!=0:
58                        ### This if statements used to check whether thumb is opened or closed
59                        if lmList[tipIds[0]][1] > lmList[tipIds[0]-1][1]:          # Checks if landmark if tip
60                            of finger is less than a certain landmark on finger along Y-axis
61                            fingers.append(1)          # 1 is added to fingers list if thumb is
62                            opened
63                        else:
64                            fingers.append(0)
65                        ### This for loop block is used to check if other 4 fingers are opened or closed
66                        for id in range(1,5):
67                            if lmList[tipIds[id]][2] < lmList[tipIds[id]-2][2]:
68                                fingers.append(1)          # Corresponding no. of 1's are added to
69                                fingers list for no. of fingers opened
70                            else:
```

```

48         fingers.append(0)
49     total=fingers.count(1)          # counts no. of 1's in fingers list and
    assigns count value to variable total
50     if total==1:                    # If no. of fingers opened at a time is 1
    then it indicates left arrow key is pressed
51         cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
52         cv2.putText(image, "BRAKE", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
53             2, (255, 0, 0), 5)
54         PressKey(brake_key_pressed)
55         brake_pressed=True
56         current_key_pressed.add(brake_key_pressed)
57         key_pressed=brake_key_pressed
58         keyPressed = True
59         key_count=key_count+1
60
61     elif total==2:                  # If no. of fingers opened at a time is 2
    then it indicates up arrow key is pressed
62         cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
63         cv2.putText(image, "JUMP", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
64             2, (255, 0, 0), 5)
65         PressKey(jump_key_pressed)
66         jump_pressed=True
67         current_key_pressed.add(jump_key_pressed)
68         key_pressed=jump_key_pressed
69         keyPressed = True
70         key_count=key_count+1
71
72     elif total==4:                  # If no. of fingers opened at a time is 4
    then it indicates down arrow key is pressed
73         cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
74         cv2.putText(image, "SLIDE", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
75             2, (255, 0, 0), 5)
76         PressKey(slide_key_pressed)
77         slide_pressed=True
78         current_key_pressed.add(slide_key_pressed)
79         key_pressed=slide_key_pressed
80         keyPressed = True
81         key_count=key_count+1
82
83     elif total==5:                  # If no. of fingers opened at a time is 5
    then it indicates right arrow key is pressed
84         cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
85         cv2.putText(image, "GAS", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
86             2, (255, 0, 0), 5)
87         PressKey(accelerate_key_pressed)
88         key_pressed=accelerate_key_pressed
89         accelerator_pressed=True
90         keyPressed = True
91         current_key_pressed.add(accelerate_key_pressed)
92         key_count=key_count+1
93     ### Implementation of code for when code is released

```

Test result

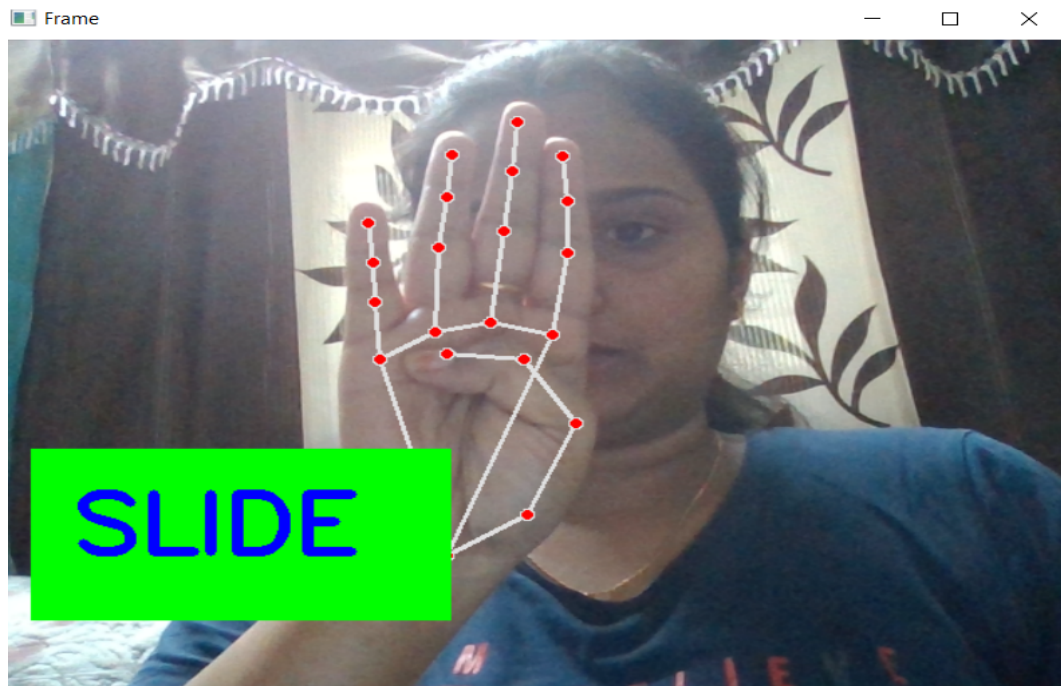


Figure 5.4: Integration Testing Result of Hand Gesture System

In figure 5.4, Integration testing result of Hand Gesture Recognition System tells that Integration testing is done in the above image. Whether the frames are detected while game is activated is tested and the name of button operated is displayed or not is verified. There are various buttons in different games to make it easy and best to understand which button is operated while people are using these names of the buttons to be displayed. All the landmarks of the hand are visible in figure and similarly the name of the button operated can also be seen.

5.3.3 System Testing - Segmentation

Input

```
1 main.py
2 import cv2
3 import mediapipe as mp
4 import time
5 from directkeys import right_pressed, left_pressed, up_pressed, down_pressed
6 from directkeys import PressKey, ReleaseKey
7
8 brake_key_pressed=left_pressed
9 accelerate_key_pressed=right_pressed
10 jump_key_pressed=up_pressed
11 slide_key_pressed=down_pressed
12 time.sleep(2.0) # suspends execution for a no. of sec
13 current_key_pressed = set() # build an unordered collection of unique elements assigns to
    a variable
14 mp_draw=mp.solutions.drawing_utils # solution for drawing utils as we are drawing utils
15 mp_hand=mp.solutions.hands # solution for hand pose
16 tipIds=[4,8,12,16,20] # List of landmarks of tip of fingers
17
18 video=cv2.VideoCapture(0) # Start capturing video from webcam, 0 refers to computer or
    default camera
19 with mp_hand.Hands(max_num_hands=1,min_detection_confidence=0.5, # max_num_hands = no. of
    hands to be detected at a time
    min_tracking_confidence=0.5) as hands:
20     # min_detection_confidence = min confidence value for detected person-model to be
    considered as successful.
21     while True: # min_tracking_confidence =
        min confidence value with which the detection from the landmark-tracking model must be
        considered as successful
22         keyPressed = False
23         brake_pressed=False
24         accelerator_pressed=False
25         jump_pressed=False
26         slide_pressed=False
27         key_count=0
28         key_pressed=0
29         ret,image=video.read() # Read video frame by frame & ret checks
        whether camera is working(1) or not(0)
30         image=cv2.cvtColor(image, cv2.COLOR_BGR2RGB ) # Convert BGR image to RGB image as opencv
        gives us BGR image
31         image.flags.writeable=False
32
33 directkeys.py
34 import ctypes
35 import time
36 SendInput = ctypes.windll.user32.SendInput
37 right_pressed=0x4D
38 up_pressed=0x48
39 down_pressed=0x50
40 left_pressed=0x4B
41 PUL = ctypes.POINTER(ctypes.c_ulong)
42 class KeyBdInput(ctypes.Structure):
43     _fields_ = [("wVk", ctypes.c_ushort),
44                 ("wScan", ctypes.c_ushort),
45                 ("dwFlags", ctypes.c_ulong),
46                 ("time", ctypes.c_ulong),
47                 ("dwExtraInfo", PUL)]
48
49 class HardwareInput(ctypes.Structure):
50     _fields_ = [("uMsg", ctypes.c_ulong),
51                 ("wParamL", ctypes.c_short),
52                 ("wParamH", ctypes.c_ushort)]
53 class MouseButton(ctypes.Structure):
54     _fields_ = [("dx", ctypes.c_long),
55                 ("dy", ctypes.c_long),
56                 ("mouseData", ctypes.c_ulong),
57                 ("dwFlags", ctypes.c_ulong),
58                 ("time", ctypes.c_ulong),
59                 ("dwExtraInfo", PUL)]
```

Test result

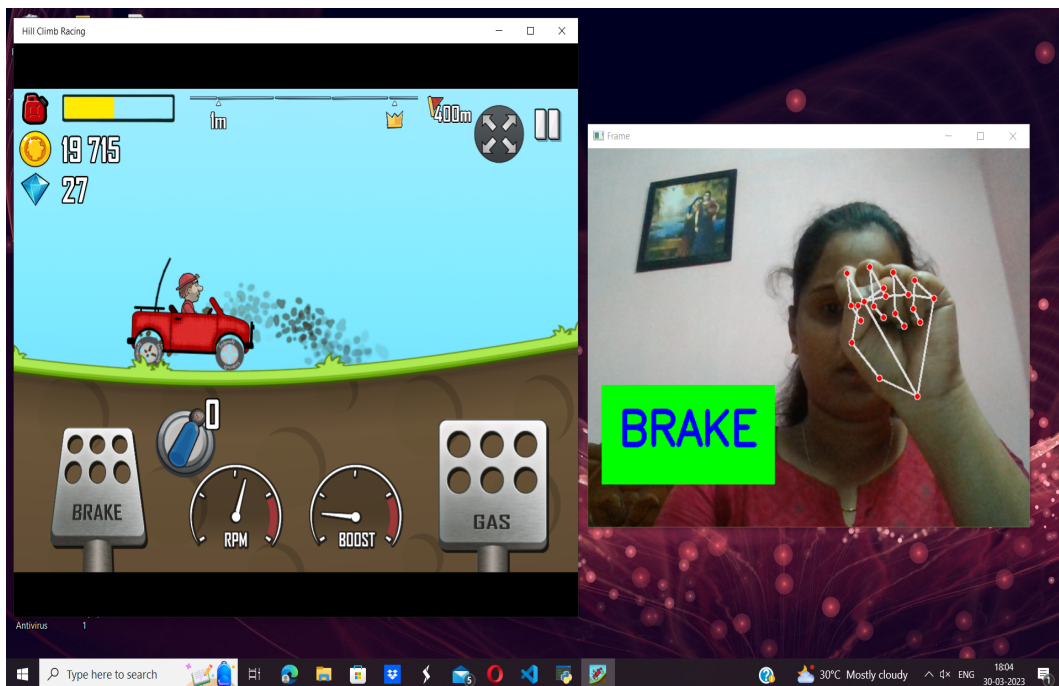


Figure 5.5: System Testing Result of Hand Gesture System

In Figure 5.5, System testing result of Hand Gesture System describes that the system testing is done. Here, it is verified whether the system is working or not and whether the game can be played using hand gesture and whether the webcam is activated and frames are detected or not are the testings done using the system here. This system is completely working and system testing is done successfully. In system testing, integration testing passed components are taken as input. The goal of integration testing is to detect any irregularity between the units that are integrated together.

Chapter 6

RESULTS AND DISCUSSIONS

6.1 Efficiency of the Proposed System

The proposed system uses a shape-based approach including several steps for hand gesture recognition, palm recognition, finger counts, etc. It is an attempt to generalize all attempts to use gesture recognition for playing games or manipulating or controlling computer systems. However, the system, at present, tends to achieve this in the domain of playing games. Compared to previously used procedures, the success rate has increased by about 22%. To make it more robust, many of the barriers in previous approaches are removed. The system aims to allow users to create customized gestures for any event/action in any domain. This is the main reason why the system cannot completely replace the computer keyboard, since it is very common for computers to be used in outdoor environments with poor lighting condition.

6.2 Comparison of Existing and Proposed System

	Classification	Accuracy
Existing Model	K-Means Algorithm & CNN	64.8%
Proposed Model	Artificial Neural Networks	86.3%

Figure 6.1: Comparison Table for Existing and Proposed System

Existing system:(K-means algorithm CNN)

In the Existing system, implemented K-means algorithm for classification of hand that is detected. When using a K-means algorithm model and CNN for classification, The system translates the detected gesture into actions such as playing games. For testing purpose, applied on Rock, Paper and Scissors, Hill Climb Racing, Off- Road Super Racing games. The games are controlled using static and dynamic gestures. Dynamic gesture is used to change controls in games. The results show that human computer interaction can be achieved even with minimal hardware requirements. gives less accurate output that is less when compared to proposed system.

A unique application that connects to the game. Furthermore, the approach of stochastic gradient descent hand gesture recognition system can efficiently track both static and dynamic hand gestures.

Proposed system:(ANN algorithm)

The proposed system deals with the problem of developing a vision based static hand gesture recognition algorithm to recognize the following four static hand gestures:left, right, up, down arrows. These gestures are chosen because they are commonly used to communicate and can thus be used in various applications, such as, a virtual Keyboard for a given application.

The feature extraction algorithms deal with problems associated with hand gesture recognition such as scaling, translation and rotation. In the classification phase where neural networks are used to recognize the gesture image based on its extracted feature, analyze some problems related to the recognition and convergence of the neural network algorithm. As a classification method, ANN has been widely employed especially for real world applications because of its ability to work in parallel. Proposed system is implemented, so that the accuracy is more when compared to the existing system. Efficiency of the Proposed system is increased by 22%.

6.3 Sample Code

```
1 import cv2
2 import mediapipe as mp
3 import time
4 from directkeys import right_pressed, left_pressed, up_pressed, down_pressed
5 from directkeys import PressKey, ReleaseKey
6
7
8 brake_key_pressed=left_pressed
9 accelerate_key_pressed=right_pressed
10 jump_key_pressed=up_pressed
11 slide_key_pressed=down_pressed
12
13 time.sleep(2.0) # suspends execution for a no. of sec
14 current_key_pressed = set() # build an unordered collection of unique elements assigns to
    a variable
15
16 mp_draw=mp.solutions.drawing_utils # solution for drawing utils as we are drawing utils
17 mp_hand=mp.solutions.hands # solution for hand pose
18
19
20 tipIds=[4,8,12,16,20] # List of landmarks of tip of fingers
21
22 video=cv2.VideoCapture(0) # Start capturing video from webcam, 0 refers to computer or
    default camera
23 ### Hands fuction of mp processes RBG image amd returns hand landmarks and handedness of detected
    hand
24 with mp_hand.Hands(max_num_hands=1,min_detection_confidence=0.5, # max_num_hands = no. of
    hands to be detected at a time
25                     min_tracking_confidence=0.5) as hands: # min_detection_confidence
    = min confidence value for detected person-model to be considered as successful.
26 while True: # min_tracking_confidence =
    min confidence value with which the detection from the landmark-tracking model must be
    considered as successful
27     keyPressed = False
28     brake_pressed=False
29     accelerator_pressed=False
30     jump_pressed=False
31     slide_pressed=False
32     key_count=0
33     key_pressed=0
34     ret,image=video.read() # Read video frame by frame & ret checks
        whether camera is working(1) or not(0)
35     image=cv2.cvtColor(image, cv2.COLOR_BGR2RGB ) # Convert BGR image to RGB image as opencv
        gives us BGR image
36     #image = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY )
37     image.flags.writeable=False # To improve the process performance we make
        model or image as not writtable
```

```

38 results=hands.process(image) # Process the RGB image only
39 image.flags.writeable=True # As completed or trained the model make
    model or image as writable
40 image=cv2.cvtColor(image, cv2.COLOR_RGB2BGR) # Convert RGB image to BGR image
41 lmList=[] # Empty list to store landmarks
42 ### Prediction made by the model is saved in the results variable from which can access
    landmarks using results.multi_hand_landmarks
43 if results.multi_hand_landmarks: # If hands are present in image(frame)
44     for hand_landmark in results.multi_hand_landmarks: # Iterate all landmarks in
        variable results
45     myHands=results.multi_hand_landmarks[0] # provides starting point
        of hand landmark i.e. 0
46     for id, lm in enumerate(myHands.landmark): # Enumerate through all ids
        and landmarks to provide x,y,z coordinates of each landmark on image
47     h,w,c=image.shape # finds height and width of the image and
        assigns to h and w
48     cx,cy= int(lm.x*w), int(lm.y*h) # finds actual integer coordinates of
        landmarks and store them in cx and cy
49     lmList.append([id,cx,cy]) # Stores each landmark values and its
        corresponding coordinates in an empty list one by one
50     mp_draw.draw_landmarks(image, hand_landmark, mp_hand.HAND_CONNECTIONS) #
        draw_landmarks function is used to draw hand landmarks and connections on image
51 fingers=[] # list to count no. of fingers visible to
    cam at a time
52 if len(lmList)!=0:
53     ### This if statements used to check whether thumb is opened or closed
54     if lmList[tipIds[0]][1] > lmList[tipIds[0]-1][1]: # Checks if landmark if tip
        of finger is less than a certain landmark on finger along Y-axis
55     fingers.append(1) # 1 is added to fingers list if thumb is
        opened
56     else:
57         fingers.append(0)
58     ### This for loop block is used to check if other 4 fingers are opened or closed
59     for id in range(1,5):
60         if lmList[tipIds[id]][2] < lmList[tipIds[id]-2][2]:
61             fingers.append(1) # Corresponding no. of 1's are added to
                fingers list for no. of fingers opened
62         else:
63             fingers.append(0)
64     total=fingers.count(1) # counts no. of 1's in fingers list and
        assigns count value to variable total

```

Output of Hand Gesture System

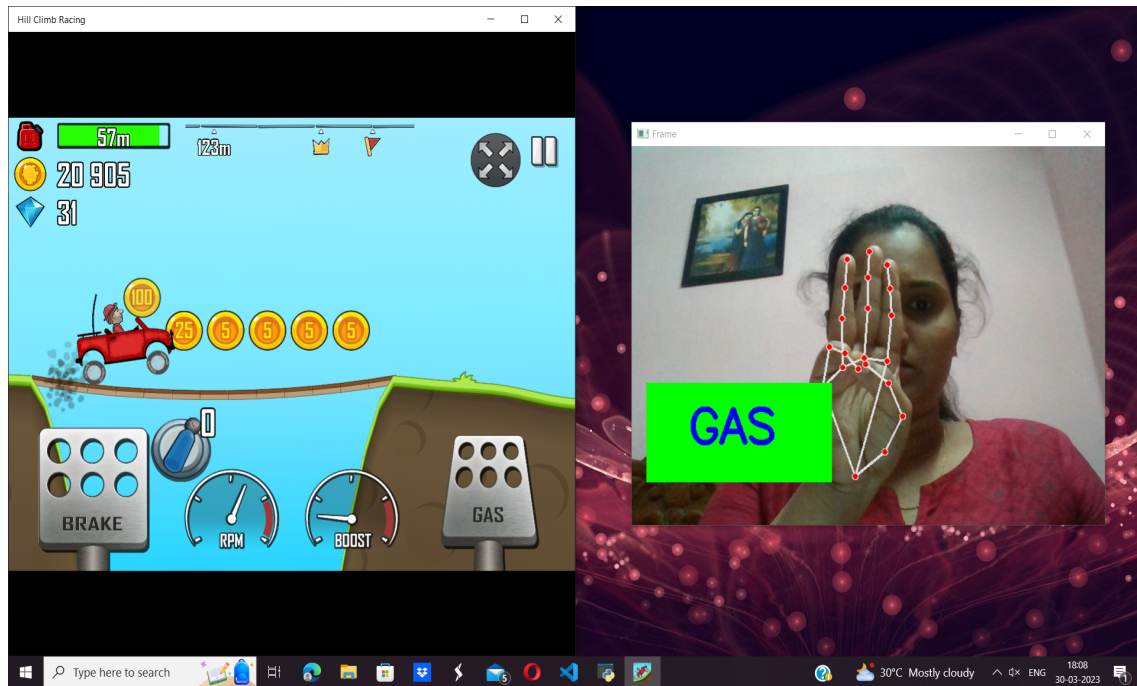


Figure 6.2: Output when Gas Button is operated

In figure 6.2, Output when Gas Button is operated describes the hand gesture shown in the above image is recognized as GAS button and the gas button in the hill climbing race is operated. The hand gesture performed to operate the GAS button is three fingers and tips of three fingers. Immediately after the three fingers are identified then the gas button is operated in the game and the car starts moving as the gas button is operated. Hence, the game is played effectively using the different hand gestures. All the landmarks are effectively identified.

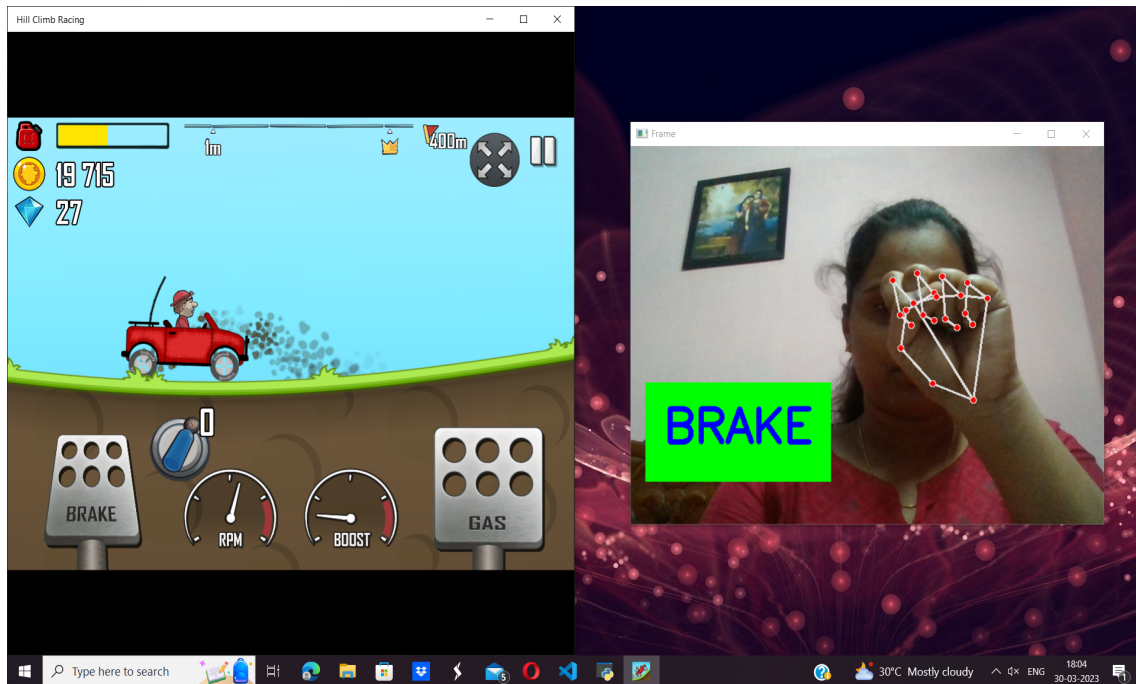


Figure 6.3: **Output when Brake Button is operated**

In figure 6.3, Output when Brake Button is operated describes that the hand gesture shown in the above image is recognized as BRAKE button and the button in the game is operated. The hand gesture performed to operate the GAS button is closed palm and tips of fingers are not needed here. Immediately after the closed palm is identified then the gas button is operated in the game and the car stops moving as the brake button is operated. Hence, the game is played effectively using the different hand gestures. So, the car starts applying the brake button here.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The machine vision-based keyboard control using hand gesture system is developed in the Python language, using the OpenCV library and Mediapipe framework. The system is able to control the movement of a keyboard key by tracking the user's hand for playing a game. The proposed machine learning algorithm used for classification is Artificial Neural Networks(ANN). The system has the potential of being a viable replacement for the computer keyboard, however due to the constraints encountered; it cannot completely replace the computer keyboard. The efficiency of capturing the input image from a live streaming video is increased(22%) compared to the existing system.

7.2 Future Enhancements

This application can be considered as a starter to the HCI application grounds, therefore large upgradations can be done to it. This application can be extended out to the mouse cursor control also using some more HCI concepts OpenCV Algorithms. Much more accuracy can be obtained with the help of Neural Networks based logics. Tracking performance can be improved to ensure better results. The accuracy of the hand gesture recognition can be improved if the template matching hand gesture recognition method is used with a machine learning classifier.

Chapter 8

INDUSTRY DETAILS

8.1 Industry name

ITC LIMITED - PAPERBOARDS SPECIALTY PAPERS DIVISION.

8.1.1 Duration of Internship (From Date - To Date)

December 13 2022- May 31 2023

8.1.2 Duration of Internship in months

6 Months

8.1.3 Industry Address

Sarapaka-507128, Burgampahad Mandal, Bhadradri Kothagudem District.

8.2 Internship offer letter

13th December 2022



ITC Limited
PAPERBOARDS & SPECIALTY PAPERS DIVISION
Unit : Bhadrachalam
Factory : SARAPAKA - 507 128,
Bhadradri Kolhagudem Dist (T.S) INDIA
Fax No : +91- 8746-242460
Telephone : +91- 8746 - 242331 - 40
www.itcpspf.com

To
Ms. Thotakura Swathi Prathyusha
M/s Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology
Chennai

Sub: Permission of Internship
Ref: Your request letter dated: 01.12.2022

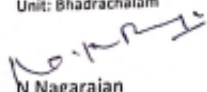
Dear Ms. Thotakura Swathi Prathyusha

With the subject and reference cited above we are pleased to accord permission for internship at our organization from 13th December 2022 to 31st May 2023. You are advised to note the following.

- 1) You will not be entitled to any remuneration, nor this internship will entitle you for any employment with the company.
- 2) You will observe all the safety rules and regulations in force such as to ensure wearing of helmet and safety shoes. The company shall, in no way be responsible for any accident during the course of internship.
- 3) You are required to prepare a detailed project report and submit a copy of the same to the undersigned.
- 4) You shall wear the Identity card issued by the company throughout the training period.
- 5) You are advised to report Mr.Arunanjan Pattanayak, IT on 13th December 2022.

Please sign in the place provided for the purpose in the copy of the letter as a token of your acceptance of the above terms and conditions from your end.

For ITC Limited
Paperboards & Specialty Papers Division
Unit: Bhadrachalam



N Nagarajan
Manager - HR

Divisional Head Quarters: 106, S.P. Road, Secunderabad - 500 003, TELANGANA
Regd. Office: ITC Limited, 37, J.L. Nehru Road, Kolkata - 700 071
FMCG, TOBACCO, HOTELS, PAPERBOARDS & PACKAGING, AGRI-BUSINESS, INFORMATION TECHNOLOGY
Corporate Website (www.itcportal.com); Corporate Identity Number : L16005WB1910PLC001985 E mail id (enduringvalue@itc.in)

Figure 8.1: Internship Offer Letter

8.3 Project Commencement Form



Project Commencement Form

Name of the Industry: ITC Paperboards and Specialty Papers Division

Address: Sarapaka-507128, Burgampahad Mandal, Bhadradri Kothagudem District, Telangana.

Team Details:

S. No	ID No	Student Name	Degree & Branch
1.	VTU13701	THOTAKURA SWATHI PRATHYUSHA	B. TECH/CSE
2.			
3.			

Date of reporting for project work: December 13

Name of the Industry Supervisor : B. Rishitha

Department : IS Department

Designation : Manager

Contact Number : 6281167362

Email ID : rishithabuyyani@gmail.com

Name of the Internal Supervisor : Mrs. S. Prema

Contact No. : 9445108754

Email ID : sprema@veltech.edu.in

Tentative Project Title / Project domain: Gaming Using Different Hand Gestures

Figure 8.2: Commencement Form

8.4 Internship Completion Certificate



ITC Limited
PAPERBOARDS & SPECIALTY PAPERS DIVISION
Unit : Bhadrachalam
Factory : SARAPAKA - 507 128,
Bhadradi Kothagudem Dist. (T.S) INDIA
Fax No. : +91- 8746-242460
Telephone : +91- 8746 - 242331 - 40
www.itcpspd.com

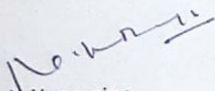
24th April 2023

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Ms. Thotakura Swathi Prathyusha, M/s Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology** has undertaken Internship at our organization in Information Technology Department. During her internship, she completed a project titled **"Gaming Using Different Hand Gestures "** at Unit Bhadrachalam, from 13th December 2022 to 24th April 2023.

We wish her all the success in future endeavors.

For ITC Limited
PSPD, Unit – Bhadrachalam


N Nagarajan
Manager – HR

Divisional Head Quarters: 106, S.P. Road, Secunderabad - 500 003. TELANGANA
Regd. Office: ITC Limited, 37, J.L. Nehru Road, Kolkata - 700 071
FMCG . TOBACCO . HOTELS . PAPERBOARDS & PACKAGING. AGRI-BUSINESS. INFORMATION TECHNOLOGY
Corporate Website (www. itcportal.com); Corporate Identity Number : L16005WB1910PLC001985 E mail id (enduringvalue@itc.in)

Figure 8.3: Internship Completion Certificate

Chapter 9

PLAGIARISM REPORT

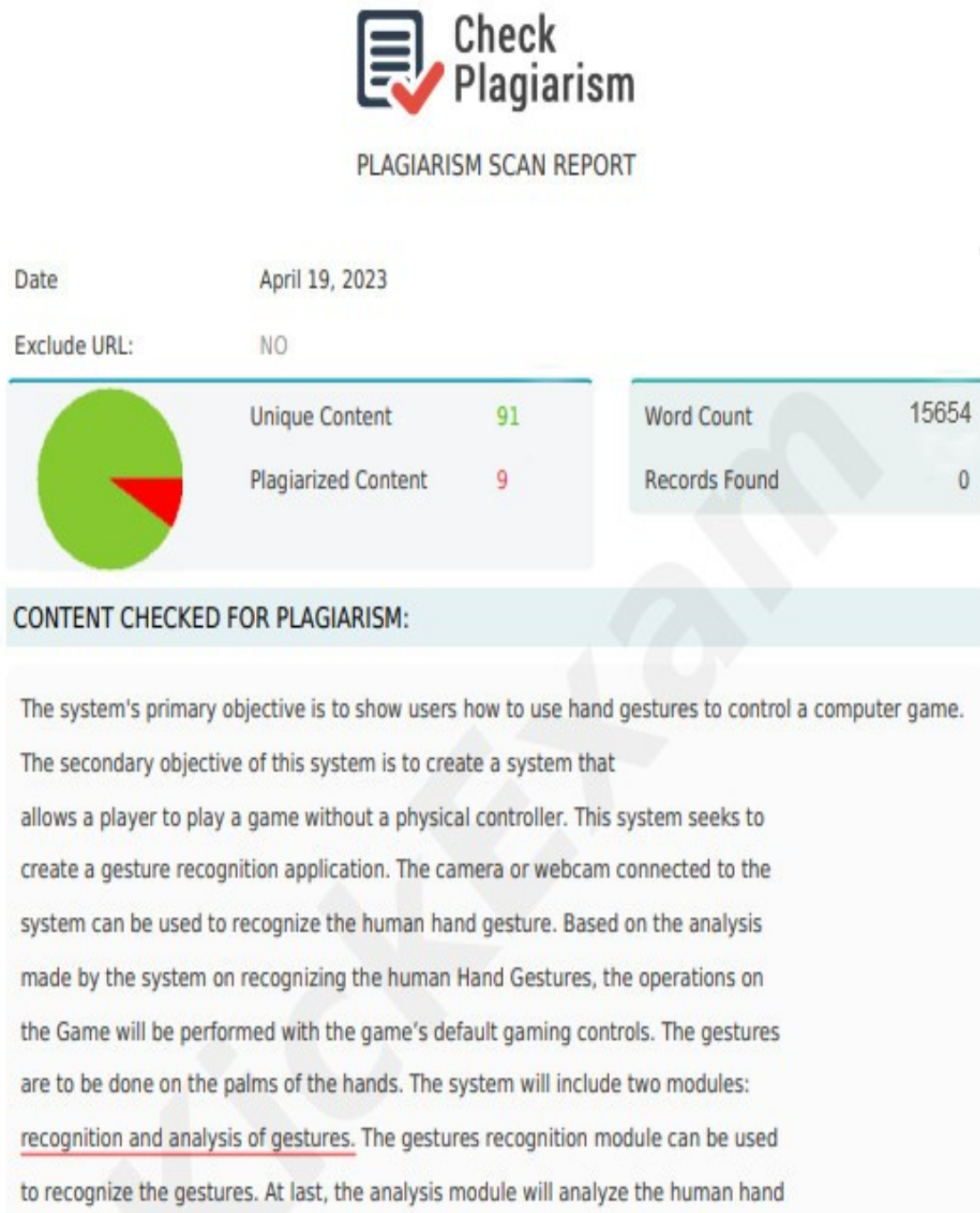


Figure 9.1: Plagiarism Report

Chapter 10

SOURCE CODE & POSTER PRESENTATION

10.1 Source Code

main.py

```
1
2 import cv2
3 import mediapipe as mp
4 import time
5 from directkeys import right_pressed, left_pressed, up_pressed, down_pressed
6 from directkeys import PressKey, ReleaseKey
7
8
9 brake_key_pressed=left_pressed
10 accelerate_key_pressed=right_pressed
11 jump_key_pressed=up_pressed
12 slide_key_pressed=down_pressed
13
14 time.sleep(2.0) # suspends execution for a no. of sec
15 current_key_pressed = set() # build an unordered collection of unique elements assigns to
    a variable
16
17 mp_draw=mp.solutions.drawing_utils # solution for drawing utils as we are drawing utils
18 mp_hand=mp.solutions.hands # solution for hand pose
19
20
21 tipIds=[4,8,12,16,20] # List of landmarks of tip of fingers
22
23 video=cv2.VideoCapture(0) # Start capturing video from webcam, 0 refers to computer or
    default camera
24 ### Hands fuction of mp processes RBG image amd returns hand landmarks and handedness of detected
    hand
25 with mp_hand.Hands(max_num_hands=1,min_detection_confidence=0.5, # max_num_hands = no. of
    hands to be detected at a time
26                     min_tracking_confidence=0.5) as hands: # min_detection_confidence
    = min confidence value for detected person-model to be considered as successful.
27 while True: # min_tracking_confidence =
    min confidence value with which the detection from the landmark-tracking model must be
```

```

considered as successful
keyPressed = False
brake_pressed=False
accelerator_pressed=False
jump_pressed=False
slide_pressed=False
key_count=0
key_pressed=0
ret,image=video.read() # Read video frame by frame & ret checks
    whether camera is working(1) or not(0)
image=cv2.cvtColor(image, cv2.COLOR_BGR2RGB ) # Convert BGR image to RGB image as opencv
    gives us BGR image
#image = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY )
image.flags.writeable=False # To improve the process performance we make
    model or image as not writable
results=hands.process(image) # Process the RGB image only
image.flags.writeable=True # As we completed or trained the model we
    make model or image as writable
image=cv2.cvtColor(image, cv2.COLOR_RGB2BGR ) # Convert RGB image to BGR image
lmList=[] # Empty list to store landmarks
### Prediction made by the model is saved in the results variable from which we can access
    landmarks using results.multi_hand_landmarks
if results.multi_hand_landmarks: # If hands are present in image(frame)
    for hand_landmark in results.multi_hand_landmarks: # Iterate all landmarks in
        variable results
        myHands=results.multi_hand_landmarks[0] # provides starting point
            of hand landmark i.e. 0
        for id, lm in enumerate(myHands.landmark): # Enumerate through all ids
            and landmarks to provide x,y,z coordinates of each landmark on image
            h,w,c=image.shape # finds height and width of the image and
                assigns to h and w
            cx,cy= int(lm.x*w), int(lm.y*h) # finds actual integer coordinates of
                landmarks and store them in cx and cy
            lmList.append([id,cx,cy]) # Stores each landmark values and its
                corresponding coordinates in an empty list one by one
        mp_draw.draw_landmarks(image, hand_landmark, mp_hand.HAND_CONNECTIONS) #
            draw_landmarks function is used to draw hand landmarks and connections on image
fingers=[] # list to count no. of fingers visible to
    cam at a time
if len(lmList)!=0:
    ### This if statements used to check whether thumb is opened or closed
    if lmList[tipIds[0]][1] > lmList[tipIds[0]-1][1]: # Checks if landmark if tip
        of finger is less than a certain landmark on finger along Y-axis
        fingers.append(1) # 1 is added to fingers list if thumb is
            opened
    else:
        fingers.append(0)
    ### This for loop block is used to check if other 4 fingers are opened or closed
    for id in range(1,5):
        if lmList[tipIds[id]][2] < lmList[tipIds[id]-2][2]:

```

```

62         fingers.append(1)                                # Corresponding no. of 1's are added to
        fingers list for no. of fingers opened
63     else:
64         fingers.append(0)
65     total=fingers.count(1)                                # counts no. of 1's in fingers list and
        assigns count value to variable total
66     if total==1:                                          # If no. of fingers opened at a time is 1
        then it indicates left arrow key is pressed
67         cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
68         cv2.putText(image, "BRAKE", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
69             2, (255, 0, 0), 5)
70         PressKey(brake_key_pressed)
71         brake_pressed=True
72         current_key_pressed.add(brake_key_pressed)
73         key_pressed=brake_key_pressed
74         keyPressed = True
75         key_count=key_count+1
76
77     elif total==2:                                        # If no. of fingers opened at a time is 2
        then it indicates up arrow key is pressed
78         cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
79         cv2.putText(image, "JUMP", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
80             2, (255, 0, 0), 5)
81         PressKey(jump_key_pressed)
82         jump_pressed=True
83         current_key_pressed.add(jump_key_pressed)
84         key_pressed=jump_key_pressed
85         keyPressed = True
86         key_count=key_count+1
87
88     elif total==4:                                        # If no. of fingers opened at a time is 4
        then it indicates down arrow key is pressed
89         cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
90         cv2.putText(image, "SLIDE", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
91             2, (255, 0, 0), 5)
92         PressKey(slide_key_pressed)
93         slide_pressed=True
94         current_key_pressed.add(slide_key_pressed)
95         key_pressed=slide_key_pressed
96         keyPressed = True
97         key_count=key_count+1
98
99     elif total==3:                                        # If no. of fingers opened at a time is 5
        then it indicates right arrow key is pressed
100         cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
101         cv2.putText(image, " GAS", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
102             2, (255, 0, 0), 5)
103         PressKey(accelerate_key_pressed)
104         key_pressed=accelerate_key_pressed
105         accelerator_pressed=True

```

```

106         keyPressed = True
107         current_key_pressed.add(accelerate_key_pressed)
108         key_count=key_count+1
109     ### Implementation of code for when code is released
110     if not keyPressed and len(current_key_pressed) != 0:
111         for key in current_key_pressed:
112             ReleaseKey(key)
113         current_key_pressed = set() # Assigning empty set or null value to
114         current_key_pressed for released key
115     elif key_count==1 and len(current_key_pressed)==2:
116         for key in current_key_pressed:
117             if key_pressed!=key:
118                 ReleaseKey(key)
119         current_key_pressed = set()
120         for key in current_key_pressed:
121             ReleaseKey(key)
122         current_key_pressed = set()
123     ### Display Video and when 'q' is entered , destroy the window
124     cv2.imshow("Frame",image) # shows the window frame
125     k=cv2.waitKey(1) # suspends frame for some sec
126     if k==ord('q'): # press q to get out of while loop or stop the code
127         break
128 video.release()
129 cv2.destroyAllWindows()

```

step.txt

```

1 import time
2 from directkeys import right_pressed , left_pressed
3 from directkeys import PressKey , ReleaseKey
4
5 break_key_pressed=left_pressed
6 accelerato_key_pressed=right_pressed
7
8 time.sleep(2.0)
9 current_key_pressed = set()
10 keyPressed = False
11 break_pressed=False
12 accelerator_pressed=False
13 key_count=0
14 key_pressed=0
15
16 cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
17 cv2.putText(image, "BRAKE", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
18             2, (255, 0, 0), 5)
19 PressKey(break_key_pressed)
20 break_pressed=True
21 current_key_pressed.add(break_key_pressed)
22 key_pressed=break_key_pressed

```

```

22         keyPressed = True
23         key_count=key_count+1
24
25
26
27         cv2.rectangle(image, (20, 300), (270, 425), (0, 255, 0), cv2.FILLED)
28         cv2.putText(image, "  GAS", (45, 375), cv2.FONT_HERSHEY_SIMPLEX,
29             2, (255, 0, 0), 5)
30         PressKey(accelerato_key_pressed)
31         key_pressed=accelerato_key_pressed
32         accelerator_pressed=True
33         keyPressed = True
34         current_key_pressed.add(accelerato_key_pressed)
35         key_count=key_count+1
36
37
38
39
40     if not keyPressed and len(current_key_pressed) != 0:
41         for key in current_key_pressed:
42             ReleaseKey(key)
43             current_key_pressed = set()
44     elif key_count==1 and len(current_key_pressed)==2:
45         for key in current_key_pressed:
46             if key_pressed!=key:
47                 ReleaseKey(key)
48             current_key_pressed = set()
49         for key in current_key_pressed:
50             ReleaseKey(key)
51         current_key_pressed = set()

```

directkeys.py

```

1 import ctypes
2 import time
3
4 SendInput = ctypes.windll.user32.SendInput
5
6
7 right_pressed=0x4D
8 up_pressed=0x48
9 down_pressed=0x50
10 left_pressed=0x4B
11
12 # C struct redefinitions
13 PUL = ctypes.POINTER(ctypes.c_ulong)
14 class KeyBdInput(ctypes.Structure):
15     _fields_ = [("wVk", ctypes.c_ushort),

```

```

16         ("wScan", ctypes.c_ushort),
17         ("dwFlags", ctypes.c_ulong),
18         ("time", ctypes.c_ulong),
19         ("dwExtraInfo", PUL)]
20
21 class HardwareInput(ctypes.Structure):
22     _fields_ = [("uMsg", ctypes.c_ulong),
23                 ("wParamL", ctypes.c_short),
24                 ("wParamH", ctypes.c_ushort)]
25
26 class MouseInput(ctypes.Structure):
27     _fields_ = [("dx", ctypes.c_long),
28                 ("dy", ctypes.c_long),
29                 ("mouseData", ctypes.c_ulong),
30                 ("dwFlags", ctypes.c_ulong),
31                 ("time", ctypes.c_ulong),
32                 ("dwExtraInfo", PUL)]
33
34 class Input_I(ctypes.Union):
35     _fields_ = [("ki", KeyBdInput),
36                 ("mi", MouseInput),
37                 ("hi", HardwareInput)]
38
39 class Input(ctypes.Structure):
40     _fields_ = [("type", ctypes.c_ulong),
41                 ("ii", Input_I)]
42
43 def PressKey(hexKeyCode):
44     extra = ctypes.c_ulong(0)
45     ii_ = Input_I()
46     ii_.ki = KeyBdInput( 0, hexKeyCode, 0x0008, 0, ctypes.pointer(extra) )
47     x = Input( ctypes.c_ulong(1), ii_ )
48     ctypes.windll.user32.SendInput(1, ctypes.pointer(x), ctypes.sizeof(x))
49
50 def ReleaseKey(hexKeyCode):
51     extra = ctypes.c_ulong(0)
52     ii_ = Input_I()
53     ii_.ki = KeyBdInput( 0, hexKeyCode, 0x0008 | 0x0002, 0, ctypes.pointer(extra) )
54     x = Input( ctypes.c_ulong(1), ii_ )
55     ctypes.windll.user32.SendInput(1, ctypes.pointer(x), ctypes.sizeof(x))
56
57 if __name__ == '__main__':
58     while (True):
59         PressKey(0x11)
60         time.sleep(1)
61         ReleaseKey(0x11)
62         time.sleep(1)

```

10.2 Poster Presentation



Vel Tech
Rangarajan Dr. Sagunthala
Vellore Institute of Technology
Chennai-600 026

GAMING USING DIFFERENT HAND GESTURES

Department of Computer Science & Engineering
School of Computing
1156CS701 – MAJOR PROJECT
WINTER SEMESTER 2022-2023

ABSTRACT

➤system's main goal is to demonstrate how a computer game may be played via hand gestures. The secondary objective of this system is to create a system that allows a player to play a game without a physical controller.

➤This system seeks to create a gesture recognition application. The camera or webcam connected to the system can be used to recognize the human hand gesture. Based on the analysis made by the system on recognizing the human Hand Gestures, the operations on the Game will be performed with the game's default gaming controls.

➤The gestures are to be done on the palms of the hands. The system will include two modules: recognition and analysis of gestures. The gestures recognition module can be used to recognize the gestures. At last, the analysis module will analyze the human hand gesture and perform game controls based on the calculated analysis.

INTRODUCTION

➤Over the past ten years, computer technology has advanced significantly and has become an indispensable aspect of daily life. The primary computer accessory for Human Computer Interaction (HCI) is the keyboard. The keyboard is not suitable for HCI in some real-life situations. The most natural and intuitive technique for HCI, that is a viable replacement for the computer keyboard is with the use of hand gestures.

➤In order to harness the full potential of a webcam, it can be used for vision-based CC, which would effectively eliminate the need for a computer keyboard. Significant Improvements in HCI gaming technology have been made during the past few decades.

➤Motion controllers are the way of the future of gaming, and they have significantly increased the sales of video games, such as the Nintendo Wii, which sold more than 50 million consoles in its first year of availability. A Natural User Interface is provided by HCI employing hand gestures, which is particularly intuitive and efficient for one-on-one computer contact (NUI). The use of hand gesture recognition in sign language recognition and HCI raises the importance of the technology even further.

RESULTS

The major constraint of the system is that it must be operated in a well-lit environment. This is the main reason why the system cannot completely replace the computer keyboard, since it is very common for computers to be used in outdoor environments with poor lighting condition.

STANDARDS AND POLICIES

➤ OpenCV-OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products OpenCV runs on the desktop operating systems: Windows, Linux, macOS Standard version: 4.7.0

➤ MEDIAPIPE: Since 2012, Google has been using it internally in several products and services. It was initially developed for real-time analysis of video and audio on We Tube. Media Pipe is a Framework for building machine learning pipelines for processing time-series data like video, audio, etc. This cross-platform Framework works in Desktop/Server, Android, iOS, and embedded devices like Raspberry Pi and Jetson Nano. Standard Used: C++11 standard.

➤ PYTHON: Python technology is quite popular among programmers, but the practice shows that business owners are also Python development believers and for good reason. It was created by Guido van Rossum and released in 1991. version of Python for Media Pipe is To make Media pipe work with TensorFlow, please set Python 3.7 as the default Python version.

METHODOLOGIES

➤STEP 1:(Pose Estimation)
Human pose estimation from video or a real-time feed plays a crucial role in various fields such as full-body gesture control, quantifying physical exercise, and sign language recognition. For example, it can be used as the base model for fitness, yoga, and dance applications. In augmented reality, it plays a significant role. A high-fidelity body pose tracking system called Media Pipe Pose uses RGB video frames as input and extrapolates 33 3D landmarks for the entire human body.

➤ STEP 2:(Preprocessing)
The first phase includes hand segmentation that aims to isolate hand gesture from the background and removing the noises using special filters. This phase includes also edge detection to find the final shape of the hand.

➤ STEP 3:(Feature Extraction)
which constitutes the main part of this research, is devoted to the feature extraction problem where two feature extraction methods, namely, hand contour and complex moments are employed. These two extraction methods were applied in this study because they used different approaches to extract the features, namely, a boundary-based for hand contour and region-based for complex moments. The feature extraction algorithms deal with problems associated with hand gesture recognition such as scaling, translation and rotation.

➤ STEP 4:(Classification)
In the classification phase where neural networks are used to recognize the gesture image based on its extracted feature, we analyze some problems related to the recognition and convergence of the neural network algorithm. As a classification method, ANN has been widely employed especially for real-world applications because of its ability to work in parallel and online training.

TEAM MEMBER DETAILS

T. Swathi Prathyusha/ VTU13701
6301797638
vtu13701@veltech.edu.in

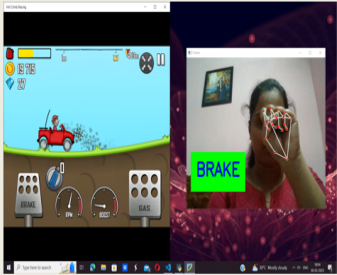
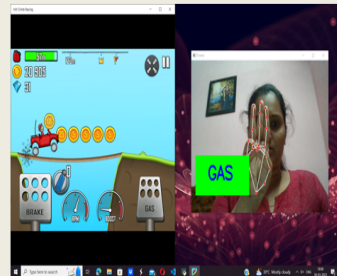



Figure 1.1: Hand gesture used while playing Hill climb game for brake

Figure 1.2: Hand gesture used while playing Hill climb game for gas

CONCLUSIONS

The proposed system uses a shape-based approach including several steps for hand gesture recognition, palm recognition, finger counts, etc. It is an attempt to generalize all attempts to use gesture recognition for playing games or manipulating or controlling computer systems. However, the system, at present, tends to achieve this in the domain of playing games. Compared to previously used procedures, the success rate has increased by about 30%. To make it more robust, many of the barriers in previous approaches are removed. The system aims to allow users to create customized gestures for any event/action in any domain

ACKNOWLEDGEMENT

1. Mrs. S. Prema/Assistant Professor
2. Project supervisor Contact No -9445108754
3. Project supervisor Mail ID – sprema@veltech.edu.in

Figure 10.1: Poster Presentation of Hand Gesture System

References

- [1] Akula Gokul, Shitanshu Ranjan Srivastava, Aditya Dixit, “Playing Games Using Hand Gesture Recognition”, International Research Journal of Modernization in Engineering Technology and Science, Volume:04, Issue:07, 2022.
- [2] Tanay Thakor, Vidya Bharde, “Hand Gesture Controlled Gaming Application” , International Research Journal of Engineering and Technology (IRJET) Volume: 08, Issue: 04, 2021.
- [3] Ahmed S. Khalaf, Ali Alshehri, “A Comparative Study of Hand-Gesture Recognition Devices for Games”, National Science Foundation government Journal, 2020.
- [4] Diego Perez-Liebana, Ahmed Khalifa, Simon M. Lucas, “General Video Game AI: a Multi-Track Framework for Evaluating Agents”, Games and Content Generation Algorithms, IEEE, Volume: 04, 2019.
- [5] Nagashree ,Dr. Parameshachari B D, Deekshith B N, “Design of an Gesture Recognition Based Car Gaming”, International Journal of Advanced Networking Applications (IJANA), ISSN: 0975-0282, 2018.
- [6] R. D. Gaina, D. Prez-Libana, and S. M. Lucas, “General Video Game for 2 Players: Framework and Competition”, in 8th Computer Science and Electronic Engineering (CEEC), IEEE, pp. 186–191, 2016.
- [7] S. Samothrakis, D. Perez-Liebana, S. M. Lucas, and M. Fasli, “Neuro evolution for General Video Game Playing”, in Conference on Computational Intelligence and Games, IEEE, pp. 200–207,2015.

- [8] D. Perez, S. Samothrakis, J. Togelius, T. Schaul, S. M. Lucas, A. Couetoux, J. Lee, C.-U. Lim, and T. Thompson, “The 2014 general video game playing competition”, *IEEE Transactions on Computational Intelligence and AI in Games*, 2015.
- [9] M. Swiechowski, J. Mandziuk, and Y. S. Ong, “Specialization of a UCT-based General Game Playing Program to Single-Player Games”, *IEEE Transactions on Computational Intelligence and AI in Games*, 2015.
- [10] D. Perez, S. Samothrakis, and S. Lucas, “Knowledge-based fast evolutionary MCTS for general video game playing”, *IEEE Conference on Computational Intelligence and Games (CIG’14)*, pp. 1–8, 2014.
- [11] J. Levine, C. B. Congdon, M. Ebner, G. Kendall, S. M. Lucas, R. Miikkulainen, T. Schaul, and T. Thompson, “General Video Game Playing”, *Dagstuhl Follow-Ups*, vol. 6, 2013.
- [12] J.H. Wu, R. Kalyanam, and R. Givan(2011), “Stochastic enforced hill- climbing”, *Journal of Artificial Intelligence Research*, pp. 815–850, 2011.
- [13] J. Mehat and T. Cazenave(2011), “A parallel general game player”, *KI- Kunstliche Intelligenz*, vol. 25, no. 1, pp. 43–47, 2011.